

Practice Exam: Answers Explained

Diagnostic Technician practice exam, Sets A and B (40 questions). Simple answers to memorize, with explanations, topology diagrams and example command output.

How each question is laid out

- Answer: in simple words, the way to write it on the exam (numbered steps for troubleshooting).
- Remember it: a short memory hook or mnemonic.
- Explanation: why the answer is right, in plain language.
- See it: a topology or diagram of what is happening.
- Try it: example command output with notes on what to look at. The values (addresses, serials, counts) are assumed, so you can see what each command prints; real servers will show their own values.

Study tip: read the answer and the explanation, then cover the answer and write it from memory using only the memory hook. The last page lists every hook for quick review.

Contents

Set A: questions 1 to 20	3
1. What servers do	4
2. What PCIe is for	6
3. Retimers versus redrivers	8
4. DHCP messages and settings	10
5. ESD risks	12
6. POST	14
7. SSH versus Serial Over LAN	16
8. BIOS and UEFI settings	18
9. ECC memory	20
10. Server will not power on	22
11. Isolation techniques	24
12. Slow NVMe drive	26
13. Failure follows the part	28
14. ip a shows 169.254	30
15. Memory: DIMM, board, CPU, or configuration	32
16. Replacement did not fix it	34
17. Filtering large logs	36

18. Network card checks by command	38
19. First commands on an unknown server	40
20. Slow server: processes and load	42
Set B: questions 21 to 40	44
21. What Linux is and why servers use it	45
22. Reading ip a output	47
23. How a name becomes an IP address	49
24. PCIe bandwidth and the math	51
25. Topology tree versus diagram	53
26. What the BMC is and does	55
27. LnkCap versus LnkSta	57
28. The four layers of a working device	59
29. Symptom versus root cause	61
30. Server lost network after a move	63
31. New card missing from lspci	65
32. Intermittent failure	67
33. Powers on but no OS	69
34. NVMe drops out under load	71
35. GPU missing, empty port in lspci -tv	73
36. Assembly cable error on three units	75
37. grep across a folder	77
38. dmesg versus journalctl -b -1	79
39. IP works but name fails, and the reverse	81
40. Saving evidence before a reboot	83
Memory hooks: one-page review	85

Set A

Questions 1 to 20

Part 1, Foundations: questions 1 to 9. Part 2, Troubleshooting: questions 10 to 16. Part 3, Linux in practice: questions 17 to 20.

What servers do

Reread: Module 1: What servers do; What makes a server different

Describe three different jobs servers do inside a company. Then explain how those jobs shape the way a server is built and managed compared with an office desktop.

 **ANSWER** (simple words: write this)

Three jobs servers do:

1. Web and application server: runs the company website and business apps that many people use at once.
2. Database or file server: stores the company's data (orders, accounts, shared files, backups) and must never lose it.
3. AI / compute server: runs heavy work like training AI models on GPUs, all day.

(Others: DNS and DHCP servers that keep the network working.)

How the jobs shape the server:

- It runs 24/7 with nobody in front of it, so it has redundancy: two power supplies and many fans. One can fail and it keeps running.
- It holds important data, so it uses ECC memory to stop silent data errors.
- It must not stop for repairs, so drives, fans and PSUs are hot-swap.
- It is managed remotely, not with a screen: a BMC, SSH and scripts. One tech can manage hundreds.
- It lives in a rack (measured in U, 1U = 1.75 in), with no keyboard or monitor.

A desktop is used by one person in office hours. If it fails, one person waits. If a server fails, the whole company can stop.

REMEMBER IT

Servers = 24/7, nobody watching. So they get R-E-H-R: Redundancy, ECC, Hot-swap, Remote management.

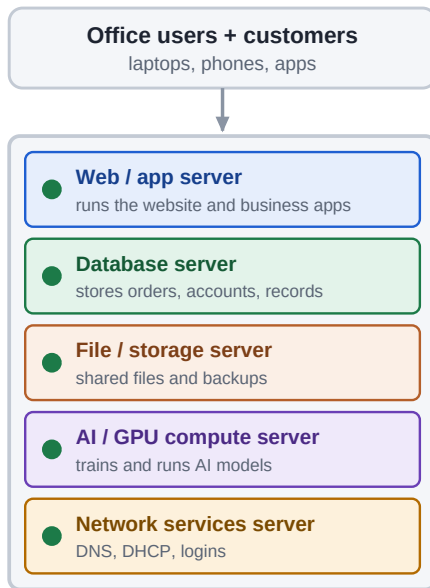
► **EXPLANATION** (understand it)

Think of a desktop as a personal car and a server as a city bus: the bus runs all day, carries many people, and has to keep going even when one part has a problem.

The job decides the design. A database server cannot lose data, so it gets ECC memory and redundant drives. A web server must stay up, so it gets two power supplies. An AI server needs huge bandwidth, so it gets GPUs on wide PCIe links. Because nobody sits at a server, everything is done remotely through the BMC and the command line.

SEE IT: TOPOLOGY / DIAGRAM

JOBS SERVERS DO



HOW THE JOB CHANGES THE MACHINE

	Office desktop	Server
Runs	office hours, a person at it	24/7, nobody in front of it
Power	1 power supply	2 hot-swap PSUs (redundant)
Cooling	a few fans	many fans + air baffles
Memory	normal RAM	ECC registered DIMMs
Managed	by the person using it	remotely: BMC, SSH, scripts
Shape	tower under a desk	rack units (1U = 1.75 in)
If it fails	one person waits	the whole company stops

Different jobs, same rule: a server is built to run all day without anyone in front of it, so it gets redundancy, ECC, hot-swap parts and remote management.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

How long has it been running? (24/7)

```
$ uptime
09:14:02 up 182 days, 4:11, 1 user, load average: 3.12, 3.40, 3.38
```

→ up 182 days: the server has run for six months without a reboot.

Redundant power, read from the BMC

```
$ sudo ipmitool sdr type "Power Supply"
PSU1 Status      | 51h | ok  | 10.1 | Presence detected
PSU2 Status      | 52h | ok  | 10.2 | Presence detected
PS Redundancy    | 77h | ok  | 7.1  | Fully Redundant
```

→ Two PSUs, and Fully Redundant: if one fails, the other carries the whole load.

What PCIe is for

Reread: Module 4: What PCIe is for; Lanes, generations, and bandwidth

What does PCIe connect inside a server, why is each link built from lanes, and how do a GPU, a network card, and an NVMe drive each depend on it?

 **ANSWER** (simple words: write this)

What it connects: PCIe is the high-speed connection between the CPU and the fast devices: GPUs, network cards (NICs), NVMe drives, and RAID or other add-in cards. Each device gets its own link back to the CPU (directly, or through a PCIe switch).

Why lanes: a link is built from lanes (x1, x4, x8, x16). Each lane is two pairs of wires: one pair sends and one pair receives, at the same time. More lanes carry more data in parallel, like more lanes on a highway. This lets the link be sized to the device.

How each device depends on it:

- GPU (usually x16): every piece of AI data goes into and out of GPU memory over PCIe. A slow or broken link means slow training or a GPU that is missing.
- Network card (x8 or x16): every network packet crosses PCIe between the NIC and the CPU. A 100G or 400G port needs a wide, fast link or it cannot reach full speed.
- NVMe drive (x4): the drive talks to the CPU directly over PCIe, with no SATA controller in between. That is why it is fast, and why PCIe problems make drives drop out or slow down.

REMEMBER IT

PCIe = the CPU's highways to the cards. Lanes = traffic lanes: more lanes, more data at once.

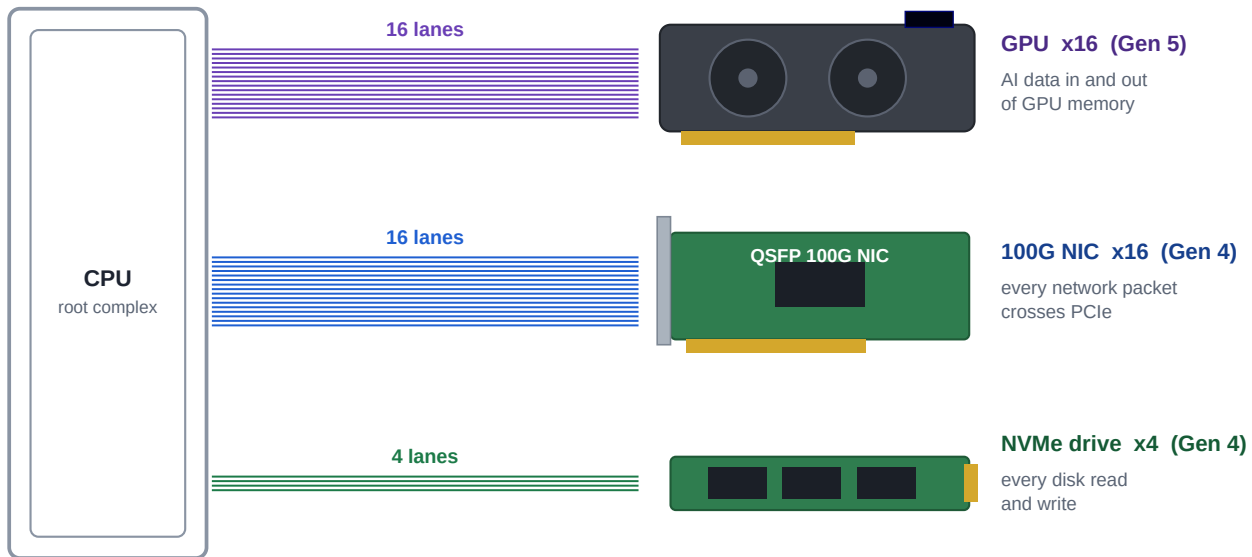
► **EXPLANATION** (understand it)

If the PCIe link to a device is missing, the device disappears from `lspci`. If the link is narrower or slower than it should be (downtrained), the device works but slowly.

Each new PCIe generation doubles the speed per lane (Gen 4 = 16 GT/s, Gen 5 = 32 GT/s). A GPU and a 100G NIC use 16 lanes each; one NVMe drive uses 4. That is why a big AI server needs a lot of lanes and often PCIe switches.

SEE IT: TOPOLOGY / DIAGRAM

PCIe: THE CPU'S HIGH-SPEED ROADS TO EVERY CARD



Each device gets its own link from the CPU. A link is made of lanes; each lane sends and receives at the same time. More lanes = more data in parallel.

>_ TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Find the three devices on the PCIe bus

```
$ lspci | grep -i -E "nvidia|ethernet|non-volatile"
19:00.0 3D controller: NVIDIA Corporation GH100 [H100 PCIe] (rev a1)
3b:00.0 Ethernet controller: Mellanox Technologies MT2892 Family [ConnectX-6 Dx]
5e:00.0 Non-Volatile memory controller: Samsung Electronics Co Ltd NVMe SSD Controller
```

→ Each line is one device on PCIe. The first part (19:00.0) is its address: bus:device.function.

How many lanes each one got

```
$ sudo lspci -vv -s 19:00.0 | grep LnkSta:
LnkSta: Speed 32GT/s, Width x16
$ sudo lspci -vv -s 3b:00.0 | grep LnkSta:
LnkSta: Speed 16GT/s, Width x16
$ sudo lspci -vv -s 5e:00.0 | grep LnkSta:
LnkSta: Speed 16GT/s, Width x4
```

→ GPU: Gen 5 x16. NIC: Gen 4 x16. NVMe drive: Gen 4 x4. Speed = generation, Width = number of lanes.

Retimers versus redrivers

Reread: Module 4: Retimers

What is a PCIe retimer, how is it different from a redriver, why are retimers so common in Gen 5 servers, and how might a failed retimer show up during testing?

 **ANSWER** (simple words: write this)

Retimer: a chip in the PCIe path that fully recovers the data and sends out a new, clean signal. This resets the "loss budget", so the signal can travel much farther. It takes part in link training, and the OS normally cannot see it (it is not in lspci). It sits on risers, backplanes, motherboards and GPU baseboards.

Redriver: a simple amplifier. It makes the signal stronger, but it also makes the noise stronger, and it does not take part in link training.

Why common in Gen 5: Gen 5 runs at 32 GT/s. Faster signals lose strength much faster over board traces, connectors, risers and cables. Long paths (to front drive bays, risers, GPU trays) cannot work at Gen 5 without a retimer, and a redriver is not good enough at that speed.

How a failed retimer shows up:

- Every device behind it disappears (a whole branch missing in `lspci -tv`).
- Links come up downtrained: `LnkSta` lower than `LnkCap`.
- A steady stream of corrected PCIe (AER) errors in `dmesg`.
- Devices that drop out under load or heat.

Because it is invisible to the OS, check the BMC (some report retimer health) and the retimer firmware version.

REMEMBER IT

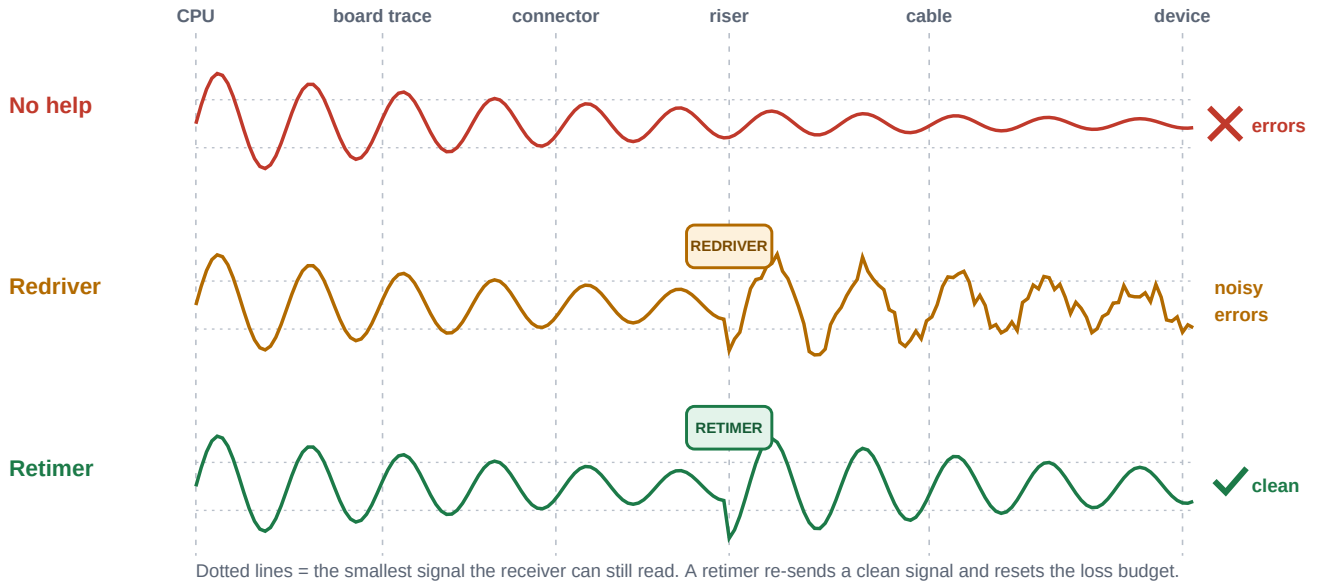
Redriver = megaphone (louder, noise too). Retimer = translator: listens, then says it again clearly.

► **EXPLANATION** (understand it)

Imagine passing a message down a long line of people. A redriver shouts the message louder, including the mistakes. A retimer listens, writes the message down correctly, then reads it out fresh, so the next part of the trip starts clean.

At Gen 3 the signal could cross a server without help. At Gen 5 the same distance loses too much signal, so retimers are everywhere. When one fails, think "everything behind it": a whole group of devices goes missing or slow together.

SEE IT: TOPOLOGY / DIAGRAM



Gen 5 signals fade fast over traces, connectors, risers and cables. A redriver amplifies everything, noise included. A retimer recovers the data and sends it again, clean.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

A weak signal: a stream of corrected PCIe errors

```
$ dmesg -T | grep -i -A3 "aer" | tail -4
[Fri Oct 2 10:41:07 2026] pcieport 0000:17:01.0: AER: Corrected error message received from 0000:19:00.0
[Fri Oct 2 10:41:07 2026] nvidia 0000:19:00.0: PCIe Bus Error: severity=Corrected, type=Physical Layer
[Fri Oct 2 10:41:07 2026] nvidia 0000:19:00.0: device [10de:2331] error status/mask=00000001/0000e000
[Fri Oct 2 10:41:07 2026] nvidia 0000:19:00.0: [ 0] RxErr
$ dmesg | grep -c "AER: Corrected"
4127
```

→ Physical Layer / RxErr = the receiver could not read the signal cleanly. Thousands of them = a signal problem on the path (seating, riser, cable, retimer).

The link also trained lower

```
$ sudo lspci -vv -s 19:00.0 | grep -E "LnkCap|LnkSta"
LnkCap: Port #0, Speed 32GT/s, Width x16, ASPM not supported
LnkSta: Speed 16GT/s (downgraded), Width x8 (downgraded)
```

→ The GPU can do Gen 5 x16 but only got Gen 4 x8. Together with the AER errors, suspect the path, including the retimer.

DHCP messages and settings

Reread: Module 7: DHCP

A brand-new server is racked and plugged in. Describe every DHCP message exchanged before it has an address, who sends each one, which are broadcasts, what settings the server ends up with, and what you would see if no DHCP server answered.

 **ANSWER** (simple words: write this)

DHCP uses four messages (DORA):

1. Discover: the new server sends "I need an address." It has no IP yet, so it is a broadcast (from 0.0.0.0 to 255.255.255.255).
2. Offer: a DHCP server answers with an offer: an IP address and settings.
3. Request: the server broadcasts "I want that offer." Broadcast, so other DHCP servers know their offers were not taken.
4. Acknowledge (ACK): the DHCP server confirms. The server sets itself up with the address.

Who sends: Discover and Request come from the client (the new server) and are broadcasts. Offer and ACK come from the DHCP server (broadcast or unicast, depending on the setup).

Broadcasts do not cross routers, so a DHCP server on another subnet needs a DHCP relay.

Settings the server ends up with: IP address, subnet mask, default gateway, DNS servers, and a lease time. The lease is renewed partway through.

If no DHCP server answers: the server keeps sending Discover, then gives up. It has no IPv4 address, or a self-assigned 169.254.x.x address. In ip a there is no normal inet line (or a 169.254 one), there is no default route, and it cannot reach anything beyond its own link.

REMEMBER IT

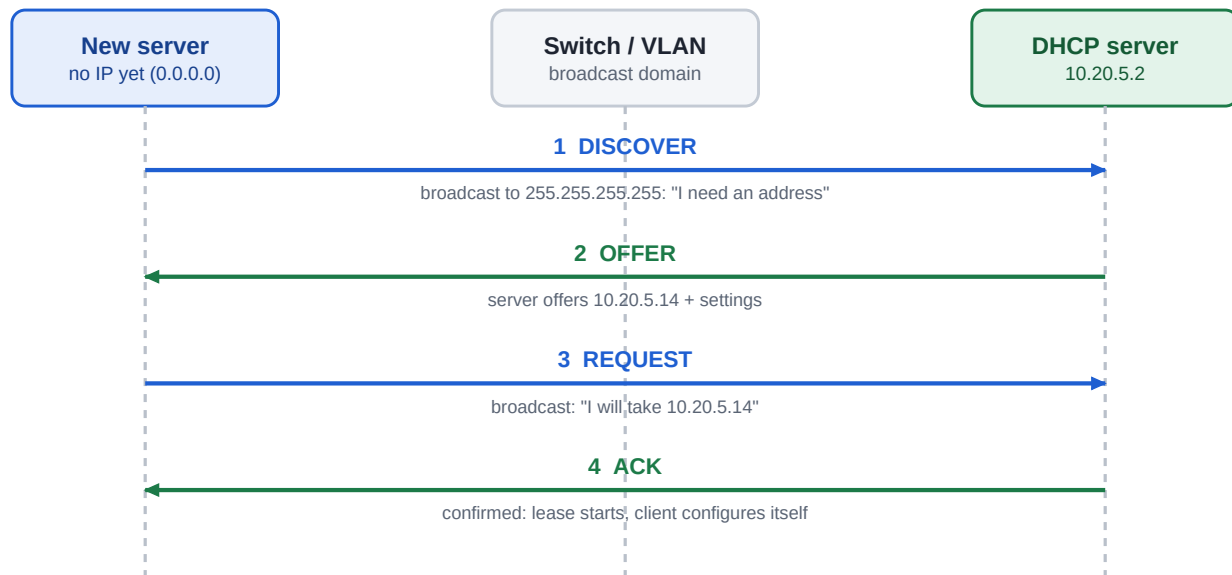
DORA: Discover, Offer, Request, Acknowledge. Settings: "I Must Get DNS Later" = IP, Mask, Gateway, DNS, Lease.

► **EXPLANATION** (understand it)

DHCP is like a hotel front desk. You walk in and shout "I need a room!" (Discover). The desk offers room 14 (Offer). You say "I'll take room 14" (Request). The desk gives you the key and the rules: Wi-Fi password, checkout time (ACK = IP, gateway, DNS, lease).

A reservation means the desk always gives the same room to the same guest (by MAC address). A static address means you never ask the desk at all.

SEE IT: TOPOLOGY / DIAGRAM



DORA: Discover and Request are broadcasts from the client; Offer and Ack come from the DHCP server. Result: IP, mask, gateway, DNS, lease time.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

A normal DHCP exchange (dhclient -v shows all four steps)

```
$ sudo dhclient -v ens1f0
Listening on LPF/ens1f0/3c:ec:ef:12:34:56
DHCPCDISCOVER on ens1f0 to 255.255.255.255 port 67 interval 3
DHCPOFFER of 10.20.5.14 from 10.20.5.2
DHCPREQUEST for 10.20.5.14 on ens1f0 to 255.255.255.255 port 67
DHCPACK of 10.20.5.14 from 10.20.5.2
bound to 10.20.5.14 -- renewal in 41413 seconds.
```

- Discover and Request go to 255.255.255.255 (broadcast). The offer comes from the DHCP server, 10.20.5.2.
- renewal in 41413 seconds: about halfway through the lease, it will renew.

When nobody answers

```
$ sudo dhclient -v ens1f0
DHCPCDISCOVER on ens1f0 to 255.255.255.255 port 67 interval 3
DHCPCDISCOVER on ens1f0 to 255.255.255.255 port 67 interval 8
DHCPCDISCOVER on ens1f0 to 255.255.255.255 port 67 interval 14
No DHCPOFFERS received.
$ ip route
(nothing: no default gateway)
```

- Only Discovers, never an Offer. Check the VLAN, the DHCP server and the relay.

Describe three different ways electrostatic discharge can harm a server part, including one that may not show up until weeks later, and the precautions that prevent them.

 **ANSWER** (simple words: write this)

ESD (electrostatic discharge) is a static zap. You often cannot feel it: you feel about 3,000 volts, but a chip can be damaged by less than 100 volts.

Three ways it harms a part:

1. Instant damage: the zap burns through part of a chip. The part is dead right away: not detected, will not power on, or fails its first test.
2. Latent damage (shows up weeks later): the zap only weakens the chip. It passes every test at the factory, then fails weeks later at the customer, after heat and use finish the damage.
3. Intermittent errors: the part works but now and then misbehaves: corrected memory errors, a PCIe or network link that drops, or random reboots. This is very hard to trace back.

Precautions:

- Wear a grounded wrist strap, connected to the chassis or an ESD mat.
- Work on a grounded ESD mat.
- Keep parts in anti-static bags until you install them.
- Hold parts by the edges; never touch pins, gold fingers or chips.
- Power off and unplug before handling parts, and use ESD-safe floors and shoes where provided.

REMEMBER IT

ESD does 3 things: Kill, Weaken, Glitch. Prevent with Strap, Mat, Bag, Edges.

► **EXPLANATION** (understand it)

Static builds up on you when you walk or move, and it jumps to the first metal it finds. A modern chip has tiny parts that a small zap can burn or weaken.

The scary one is latent damage: the unit passes test and ships, then fails weeks later. You saved nothing by skipping the wrist strap; you just moved the failure to the customer.

SEE IT: TOPOLOGY / DIAGRAM

THREE WAYS ESD HURTS A PART



1 Instant damage

The zap burns through the chip. The part is dead now: not detected, will not power on, fails its first test.

2 Latent damage (shows up weeks later)

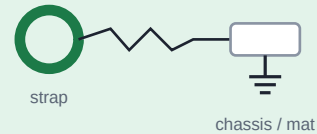


3 Intermittent errors

It works, but now and then: corrected memory errors, a link that drops, a random reboot. Very hard to trace.



Prevent it



- ✓ Wear a grounded wrist strap
- ✓ Work on an ESD mat
- ✓ Keep parts in anti-static bags
- ✓ Hold parts by the edges
- ✓ Never touch pins or gold fingers
- ✓ Power off and unplug first

You feel ~3,000 V. Chips can die from < 100 V.

ESD can kill a part at once, weaken it so it fails weeks later, or leave it working with random errors. You cannot feel a zap that is big enough to damage a chip.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

What latent damage can look like weeks later (one possible cause)

```
$ sudo ipmitool sel elist | tail -3
41 | 11/06/2026 | 02:14:51 | Memory DIMM_B1 | Correctable ECC | Asserted
44 | 11/09/2026 | 17:30:02 | Memory DIMM_B1 | Correctable ECC | Asserted
4a | 11/12/2026 | 08:11:47 | Memory DIMM_B1 | Correctable ECC logging limit reached | Asserted
```

→ This DIMM passed every test at build time in October. Errors start in November and grow. Handling damage is one possible cause, which is why the wrist strap matters every time.

A large server stays on a blank screen for several minutes after power on before anything appears. Explain what the firmware is doing during that time, and why skipping those checks would be risky.

 ANSWER (simple words: write this)

During the blank screen, the firmware (BIOS/UEFI) runs POST, the Power-On Self Test. It checks and sets up the hardware in order:

1. Power and BMC: the power supplies come up and the BMC hands over to the firmware.
2. CPU initialization: starts the CPUs and loads microcode.
3. Memory training and testing: tunes the timing of every memory channel for every DIMM, then tests the memory. On a server with hundreds of GB or several TB of memory, this takes minutes. It is the longest step.
4. PCIe enumeration: finds every card and trains every PCIe link (speed and width).
5. Option ROMs: starts the firmware on RAID cards, NICs and PXE.
6. Finds the boot device and hands over to the bootloader.

The screen stays blank because video starts late in this process. Be patient: several minutes is normal on a large server. Watch the POST code or the BMC to see progress.

Why skipping it is risky:

- Memory that is not trained and tested can give random errors, data corruption and crashes later in the OS.
- A bad DIMM, CPU or card would not be caught; the server would boot and fail later, maybe at the customer.
- PCIe links that are not trained properly leave cards missing or slow.

REMEMBER IT

POST = the server's morning check-up: CPU, Memory, PCIe, ROMs, Boot. Memory training is the long part.

► **EXPLANATION** (understand it)

Memory training is like tuning a radio for every station: the firmware adjusts timing on each channel until the signal is clean. More DIMMs = more tuning = longer wait. A server with 2 TB of memory can take five minutes or more.

If a server stays blank much longer than normal, check the POST code and the BMC event log: that tells you where it stopped (memory, CPU or a PCIe card).

SEE IT: TOPOLOGY / DIAGRAM

WHAT THE FIRMWARE DOES BEFORE ANYTHING SHOWS ON SCREEN (POST)



If you skipped these checks...

- Memory not trained or tested: random errors, data corruption, crashes later in the OS.
- A bad DIMM, CPU or card is not caught: the server boots and fails at the customer instead.
- PCIe links not trained: cards missing or slow (downtrained).
- **Be patient: on a server with lots of memory, minutes of blank screen is normal. Watch the POST code / BMC.**

POST checks and sets up the hardware in order. Memory training is the long part, and it grows with the amount of memory installed.

>_ TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Watching POST progress in the BMC event log

```
$ sudo ipmitool sel elist | tail -6
1b | 10/02/2026 | 08:00:05 | System Firmware Progress | Primary CPU initialization | Asserted
1c | 10/02/2026 | 08:00:19 | System Firmware Progress | Memory initialization | Asserted
1d | 10/02/2026 | 08:04:52 | System Firmware Progress | PCI resource configuration | Asserted
1e | 10/02/2026 | 08:05:20 | System Firmware Progress | Option ROM initialization | Asserted
1f | 10/02/2026 | 08:05:31 | System Firmware Progress | Video initialization | Asserted
20 | 10/02/2026 | 08:05:44 | System Firmware Progress | Starting operating system boot process | Asserted
```

→ Memory took 08:00:19 to 08:04:52: about 4.5 minutes.

→ Video starts at 08:05:31. Before that the screen is blank, which is normal.

Why it took so long

```
$ free -h | head -2
      total        used        free      shared  buff/cache   available
Mem:    2.0Ti       38Gi       1.9Ti        1.1Gi        12Gi        1.9Ti
```

→ 2 TB of memory to train and test.

SSH versus Serial Over LAN

Reread: Module 8: SSH versus Serial Over LAN

A coworker says: “Serial Over LAN is just a slower version of SSH.” Explain why that is wrong. Describe how each one reaches the server, what each depends on, and give one situation where only SOL will help.

 **ANSWER** (simple words: write this)

It is wrong because SSH and SOL reach the server by completely different roads and show different things.

SSH (in-band):

- Runs inside the operating system, over the server's normal network, to the sshd service on TCP port 22.
- Needs: the OS booted, the network working, and sshd running.
- You get a login shell, only after the OS is up.

SOL, Serial Over LAN (out-of-band):

- Goes through the BMC over the management network, to the server's serial console.
- Needs only the BMC, which runs on standby power. It does not need the OS.
- You see the console: BIOS/POST (if console redirection is on), the GRUB menu, kernel boot messages, and kernel panics.

So SOL is not "slow SSH". It is a different door into the server.

Situation where only SOL helps: a server boots into emergency mode because of a bad line in `/etc/fstab`. The network never starts, so SSH fails. With SOL (`ipmitool sol activate`) I can watch the boot, log in at the emergency prompt and fix `fstab`.

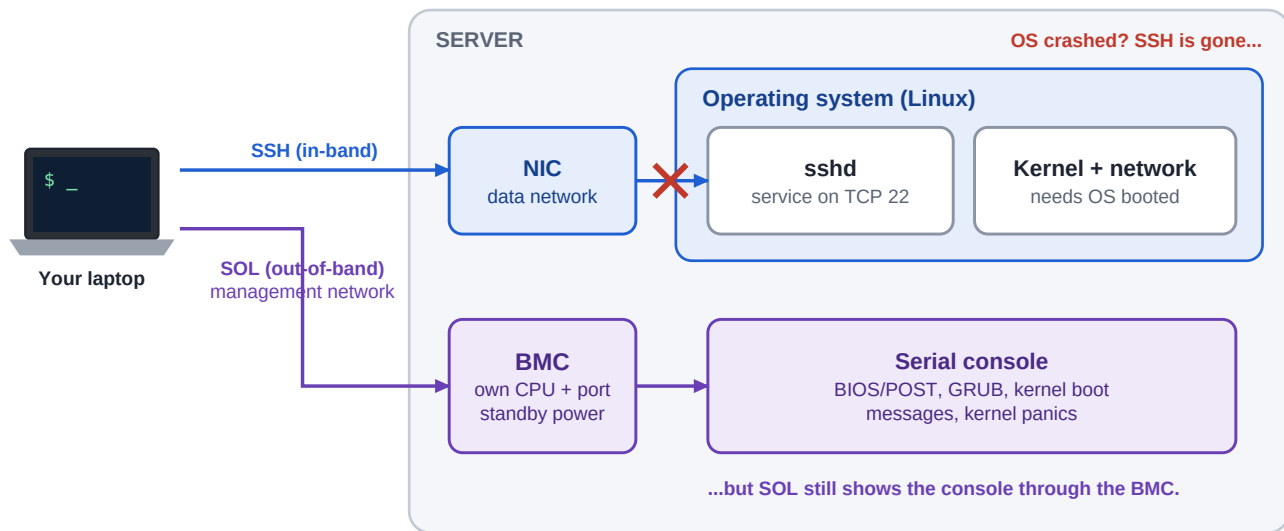
REMEMBER IT

SSH talks to the OS. SOL talks to the console through the BMC. OS broken = SSH gone, SOL still works.

► **EXPLANATION** (understand it)

SSH is like phoning someone at their desk: it only works if they are at work and the phone line is up. SOL is like the building's intercom from the security desk: it works even when the office is closed.

The virtual console (KVM) in the BMC web page is the graphical version of SOL. Use SSH for daily work on a healthy server; use SOL or KVM when the OS is broken, frozen, or has lost its network.



SSH reaches the operating system over the data network. SOL reaches the console through the BMC. Different roads, so SOL is not "slower SSH".

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

SSH fails: the OS never brought up its network

```
$ ssh tech@10.20.5.14
ssh: connect to host 10.20.5.14 port 22: No route to host
```

→ SSH needs the OS and its network. Neither is up.

SOL still works, through the BMC

```
$ ipmitool -I lanplus -H 10.20.9.14 -U admin -P ***** sol activate
[SOL Session operational. Use ~? for help]
[ TIME ] Timed out waiting for device /dev/disk/by-uuid/9f1c2a7e...
[DEPEND] Dependency failed for /data.
You are in emergency mode. After logging in, type "journalctl -xb" to view
system logs, "systemctl reboot" to reboot, or "exit" to boot into default mode.
Give root password for maintenance
```

→ 10.20.9.14 is the BMC address on the management network, not the server's OS address.

→ SOL shows the boot itself, and the exact reason it stopped.

BIOS and UEFI settings

Reread: Module 9: Firmware; Module 4: PCIe errors and BIOS settings

List four kinds of settings stored in a server's BIOS or UEFI setup. For each one, describe a problem a wrong setting could cause.

 **ANSWER** (simple words: write this)

1. Boot settings: boot order, UEFI or Legacy mode, Secure Boot, PXE (network boot).

Wrong setting: "No bootable device", booting from the network in a loop (PXE loop), or the OS will not load because the mode is wrong.

2. PCIe settings: slot enabled or disabled, bifurcation (splitting an x16 slot into x4 links), link speed, Above 4G Decoding, Resizable BAR.

Wrong setting: a card does not appear in lspci, a link runs at x4 instead of x16, an NVMe adapter shows only one drive, or a large GPU will not start.

3. Memory settings: memory speed, memory mode (mirroring or sparing), patrol scrub.

Wrong setting: the OS shows less memory than installed (mirroring hides half), or the server is unstable at a speed the DIMMs do not support.

4. CPU and power settings: virtualization (VT-x/VT-d), number of cores, Hyper-Threading, power profile.

Wrong setting: virtual machines will not start (virtualization off), or the server is slow because a power-saving profile limits the CPU.

Also: console redirection (serial) and BMC network settings. If console redirection is off, SOL shows nothing during POST.

REMEMBER IT

BIOS settings: Boot, PCIe, Memory, CPU/Power (+ Console). "Big PCs Make Commotion".

► **EXPLANATION** (understand it)

The BIOS/UEFI setup is the server's "control panel" before the OS starts. It is stored in firmware, so it stays even when the OS is reinstalled. A wrong setting looks like a hardware fault: the card is fine, but the slot is switched off.

That is why a good technician checks BIOS settings before replacing parts, and records BIOS versions and settings when comparing with a golden unit.

SEE IT: TOPOLOGY / DIAGRAM

Server Setup Utility (UEFI)

Main	Boot	PCIe	Memory	CPU/Power	Mgmt
1	Boot Mode				[UEFI]
1	Boot Option #1			[NVMe: PM9A3 960G]	
1	Secure Boot			[Enabled]	
2	Slot 3 Enable			[Enabled]	
2	Slot 3 Bifurcation			[x4x4x4x4]	
2	Link Speed			[Auto]	
2	Above 4G Decoding			[Enabled]	
2	Resizable BAR			[Enabled]	
3	Memory Mode			[Independent]	
3	Memory Speed			[Auto]	
4	VT-x / VT-d			[Enabled]	
4	Power Profile			[Performance]	
5	Console Redirect			[COM1 115200]	

1 Boot settings

boot order, UEFI/Legacy, Secure Boot

Wrong: "No bootable device" or a PXE loop

2 PCIe settings

slot on/off, bifurcation, speed, 4G, BAR

Wrong: card missing, x4 not x16, GPU fails

3 Memory settings

speed, mirroring, sparing

Wrong: less memory shown, unstable

4 CPU / power

virtualization, cores, power profile

Wrong: VMs fail, slow server (power saving)

5 Console / mgmt

serial redirect, BMC network

Wrong: SOL shows nothing during POST

The BIOS/UEFI setup stores how the hardware starts. One wrong setting can hide a device, slow the server, or stop it booting.

>_ TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Checking a few BIOS effects from Linux

```
$ sudo dmidecode -s bios-version
2.4b
$ lscpu | grep -i virtualization
Virtualization:          VT-x
$ mokutil --sb-state
SecureBoot enabled
```

- BIOS version: compare it with the approved version.
- If VT-x were disabled in the BIOS, the Virtualization line would be missing and VMs would fail.
- Secure Boot state comes from the UEFI setting.

A PCIe setting that hides devices: bifurcation

```
$ sudo nvme list | grep -c nvme
1
```

- A 4-drive NVMe adapter in a slot that is NOT bifurcated (x16 instead of x4x4x4x4) shows only one drive. The drives are fine; the setting is wrong.

A manager asks why servers use ECC memory when most desktops do not. Explain how ECC works, what it can and cannot fix, and why the log of corrected errors matters.

 **ANSWER** (simple words: write this)

How ECC works: ECC (Error-Correcting Code) memory stores extra check bits with the data. When the data is read back, the memory controller uses the check bits to see if anything changed.

What it can fix: a single-bit error. One flipped bit is found and fixed on the spot. This is a correctable error: the server keeps running and logs it.

What it cannot fix: a double-bit error. It is detected but cannot be fixed. This is an uncorrectable error, and the server usually stops or crashes on purpose rather than use bad data. ECC also cannot save a DIMM that has completely failed.

Why servers need it (and desktops usually do not): servers run 24/7 with huge amounts of memory, so bit flips will happen. Without ECC, a flipped bit causes silent data corruption or a random crash, and nobody knows why. For a database server, wrong data is much worse than a crash.

Why the corrected-error log matters: it is an early warning. A DIMM that logs more and more corrected errors is likely to fail. You see it in the BMC event log (e.g. "Correctable ECC logging limit reached, DIMM C1"), `dmesg | grep -i edac`, or `ras-mc-ctl --summary`. Then you replace the DIMM in a planned window, before it causes an uncorrectable error and an outage.

REMEMBER IT

ECC: 1 bit = fix it, 2 bits = flag it (stop). A rising corrected count = replace soon.

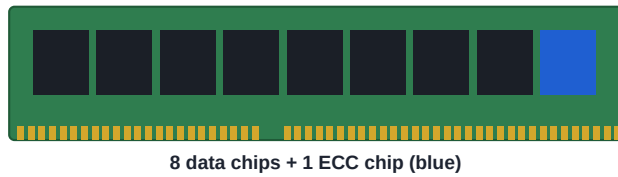
► **EXPLANATION** (understand it)

ECC is like a spell-checker with one rule: it can fix one wrong letter in a word, and it can tell you a word has two wrong letters, but it cannot fix that word.

Corrected errors are not "fine because they were fixed". They are a warning light: the DIMM is wearing out or something (seating, the slot, the CPU) is not right. Track the count over time.

SEE IT: TOPOLOGY / DIAGRAM

AN ECC RDIMM HAS A 9TH CHIP FOR CHECK BITS



WHAT ECC CAN AND CANNOT DO

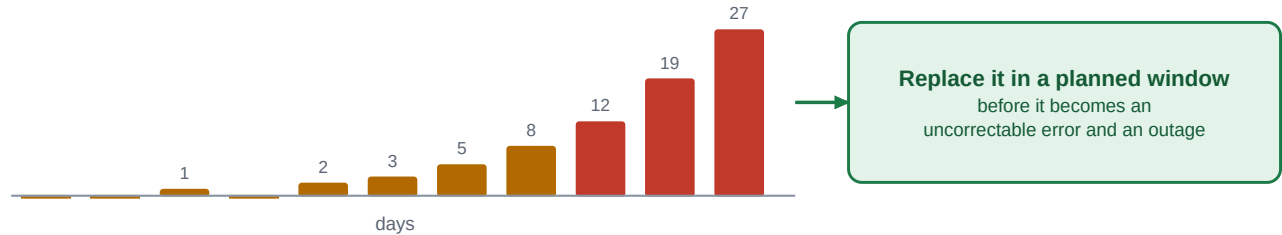
1 bit flipped: correctable

1 0 0 0 0 0 1 0 fixed, logged

2 bits flipped: uncorrectable

1 0 1 1 0 1 1 0 detected, crash

CORRECTED ERRORS PER DAY ON DIMM C1: THE EARLY WARNING



ECC: one bit fixed, two bits flagged. A rising count of corrected errors means the DIMM is wearing out.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

The BMC event log: the warning, then the limit

```
$ sudo ipmitool sel elist | grep -i ecc
51 | 09/28/2026 | 03:12:44 | Memory DIMM_C1 | Correctable ECC | Asserted
5c | 10/01/2026 | 22:40:19 | Memory DIMM_C1 | Correctable ECC logging limit reached | Asserted
```

→ Same DIMM, more and more errors, until the BMC stops logging each one. Plan a replacement.

The Linux side

```
$ sudo ras-mc-ctl --summary
Memory controller events summary:
    Corrected on DIMM Label(s): 'CPU_SrcID#0_MC#0_Chan#2_DIMM#0' location: 0:2:0:-1 errors: 37
$ dmesg | grep -i edac | tail -1
[812345.120411] EDAC MC0: 1 CE memory read error on CPU_SrcID#0_MC#0_Chan#2_DIMM#0 (channel:2 slot:0 ...)
```

→ CE = Corrected Error. 37 so far on one DIMM. Map the label to a slot and serial with `dmidecode -t memory`.

Server will not power on

Reread: Module 9: A server that will not power on

A server shows no lights and does not start from its front panel, while the server above it on the same PDU runs normally. Describe your checks in order, and explain how the BMC helps you decide where the problem is.

 ANSWER (simple words: write this)

The server above works on the same PDU, so the PDU itself has power. The problem is between this server's outlet and its board.

1. Check the PDU outlet for this server: is that outlet switched on, or did its breaker or outlet group trip? Is the cord in the right outlet?
2. Check both power cords: seated at the PDU and at the PSUs, not damaged. Try a known-good cord and a known-good outlet.
3. Check the PSU LEDs: is there an AC/standby light on each PSU? Reseat each PSU.
4. Ask the BMC (ping, web page or ipmitool):
 - If the BMC answers, standby power is reaching the board, so the cords and outlet are OK. Look at the power-on path: chassis status (Last Power Event, faults), the SEL (PSU or power events), try ipmitool power on (if that works, suspect the front panel button or its cable).
 - If the BMC is dead too, there is no standby power at all: focus on the outlet, cords and PSUs.
5. Swap a PSU with a known-good one.
6. If the PSUs are fine but it still will not start, suspect the board or a short: remove cards and extra parts and try a minimum configuration.
7. Record what you found, verify it powers on and passes test, and document.

REMEMBER IT

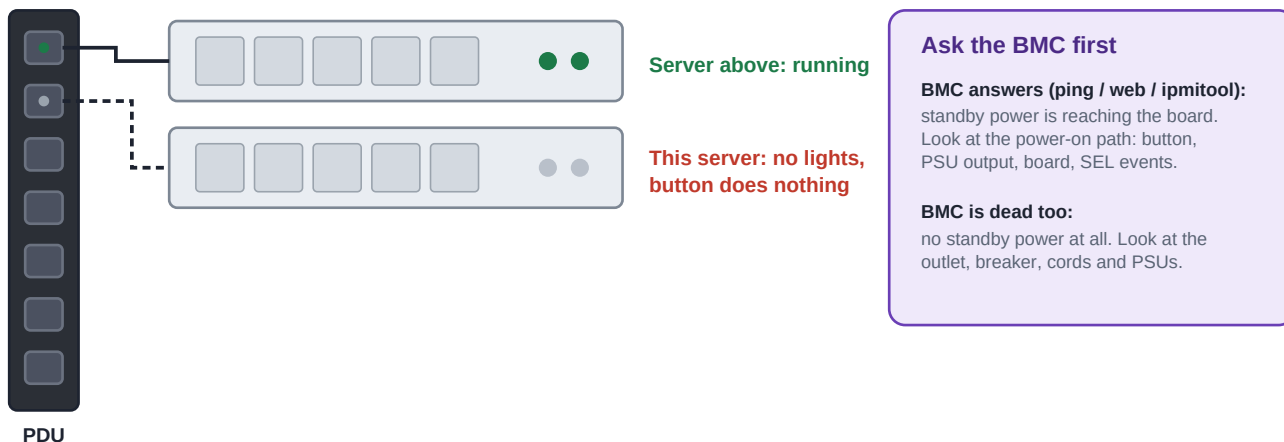
Follow the power: Outlet, Cord, PSU, Standby/BMC, Button, Board. The BMC tells you if standby power arrives.

► **EXPLANATION** (understand it)

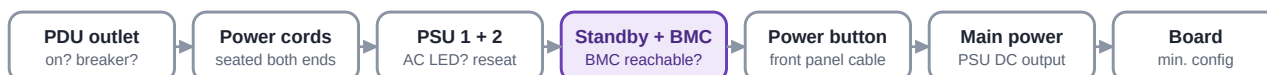
Power reaches a server in a chain: PDU outlet, cord, PSU, standby power to the board (which runs the BMC), then main power when the button is pressed. Check the links in that order, one at a time.

The BMC is your "is standby power arriving?" test. A BMC that answers proves the outlet, cord and at least one PSU are working, and its logs often show exactly what happened (for example AC lost, a PSU fault, or a power policy that keeps it off).

SEE IT: TOPOLOGY / DIAGRAM



FOLLOW THE POWER, ONE LINK AT A TIME



The server above works on the same PDU, so the PDU has power. Check this server's outlet, cords and PSUs, and use the BMC to see whether standby power arrives.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

The BMC answers, so standby power is there

```
$ ping -c 2 10.20.9.15
64 bytes from 10.20.9.15: icmp_seq=1 ttl=64 time=0.41 ms
$ ipmitool -I lanplus -H 10.20.9.15 -U admin -P ***** chassis status
System Power      : off
Power Overload    : false
Main Power Fault  : false
Power Control Fault : false
Power Restore Policy : always-off
Last Power Event   : ac-failed
```

- ac-failed: it lost AC power (outlet, breaker or cord).
- always-off: when power came back, the BMC kept it off on purpose. That is why it looks dead.

The event log confirms it, then power on

```
$ ipmitool -I lanplus -H 10.20.9.15 -U admin -P ***** sel elist | tail -2
8a | 10/02/2026 | 06:12:30 | Power Supply PSU1 | Power Supply AC lost | Asserted
8b | 10/02/2026 | 06:12:30 | Power Supply PSU2 | Power Supply AC lost | Asserted
$ ipmitool -I lanplus -H 10.20.9.15 -U admin -P ***** power on
Chassis Power Control: Up/On
```

- Both PSUs lost AC at the same second: the outlet or its breaker, not the PSUs. Find out why before you close the ticket.

Isolation techniques

Reread: Module 10: Isolation techniques

Describe four techniques for proving which hardware part is causing a failure. For each, explain what result would convince you, and one risk or limitation.

 **ANSWER** (simple words: write this)

1. Swap with known-good: move the suspect part to a known-good slot or system, or put a known-good part in the suspect location.

Convinced when: the failure follows the part to its new place (the part is bad), or stays with the slot when a good part is there (the slot, cable or board is bad), and it repeats.

Risk: a bad slot or a power problem can damage the good part you put in. Inspect the slot first.

2. Minimum configuration: strip the server to the minimum (one CPU, one DIMM, no cards), then add parts back one at a time.

Convinced when: it works at minimum and fails the moment one particular part is added back.

Limit: slow, and a fault that only shows under full load may not appear with a minimal setup.

3. Half-splitting: split the possible causes into two halves and test which half has the problem; keep splitting.

Convinced when: the fault stays inside one half every time you split, down to one part.

Limit: you need parts you can test separately; it is hard when everything depends on everything.

4. Compare with a golden unit: compare the failing unit with a known-good identical unit: logs, firmware versions, BIOS settings, cabling, test results.

Convinced when: you find the one difference, and changing it on the failing unit fixes it.

Limit: the golden unit must really be identical and really good.

In all four, change only one thing at a time and document each result.

REMEMBER IT

Four ways to catch the bad part: Swap, Strip, Split, Compare.

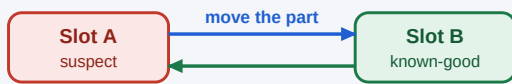
► **EXPLANATION** (understand it)

All four techniques do the same thing: they change one thing on purpose and watch what happens. "Follows the part" and "stays with the slot" are the two answers a swap can give you.

Say the risk in your answer. Graders like to see that you know a swap can kill a good part, and that a golden unit is only useful if it truly matches.

SEE IT: TOPOLOGY / DIAGRAM

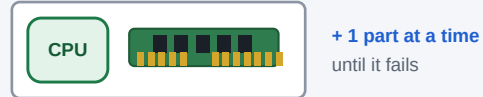
1 Swap with known-good



Convinced: fault follows the part (or stays with the slot) every time.

Risk: a bad slot can damage the good part.

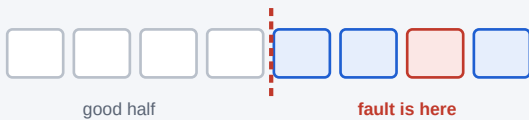
2 Minimum configuration



Convinced: it fails the moment one part is added back.

Limit: slow; a load-only fault may not appear.

3 Half-splitting



Convinced: the fault stays in one half as you keep splitting.

Limit: needs parts you can test separately.

4 Compare with a golden unit



Convinced: you find the one difference, and changing it fixes it.

Limit: the golden unit must really match.

Four ways to prove the faulty part. Each one changes only one thing at a time, and each has a limit you should say out loud in your answer.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Golden unit comparison finds the difference

```
$ diff golden_unit_versions.txt J7K2Q41_versions.txt
3c3
< BIOS Version : 2.4b
---
> BIOS Version : 2.1
7c7
< NIC FW      : 22.36.1010
---
> NIC FW      : 22.31.1014
```

→ Lines with < are the golden unit, > the failing one. Two differences: update one, retest, then the other.

Swap log: does the fault follow the part?

```
$ cat swap_log.txt
GPU SN ...4417 in slot 3 -> FAIL (Xid 79)
GPU SN ...4417 in slot 5 -> FAIL (Xid 79)  <- moved the GPU
GPU SN ...8810 in slot 3 -> PASS           <- known-good GPU in slot 3
```

→ The fault followed GPU ...4417 to slot 5, and slot 3 passes with a good GPU: the GPU is bad.

Slow NVMe drive

Reread: Module 6: A drive that is slower than it should be

Two identical NVMe drives in the same server are tested the same way. One reaches its rated speed; the other gets about half. List the possible causes and how you would check each one.

ANSWER (simple words: write this)

1. Downtrained link: the slow drive may have trained at x2 or Gen 3 instead of Gen 4 x4. Check `sudo lspci -vv` on both drives and compare `LnkSta` with `LnkCap`. Half the lanes = about half the speed.
2. A different path: the slow drive may share a PCIe switch uplink with other busy devices, or sit behind the other CPU. Check `lspci -tv` against the topology diagram.
3. Heat: the drive may be throttling because its bay is hot. Check `nvme smart-log` for temperature and thermal throttling counts, and the BMC sensors.
4. Different firmware: check `nvme list`; the firmware version should match.
5. Drive state or format: a nearly full drive, a different sector format (`nvme id-ns`), or one that was never trimmed is slower. Check usage and format.
6. The test is not really the same: wrong device, different queue depth or block size, or another program using the drive. Re-run the identical `fio` command on both.
7. Signal problems or a failing drive: check `dmesg` for AER errors and `nvme smart-log` for media errors and warnings.

Then isolate: swap the two drives between bays. If the slowness follows the drive, it is the drive; if it stays with the bay, it is the bay, backplane, cable or path. Change one thing at a time and document.

REMEMBER IT

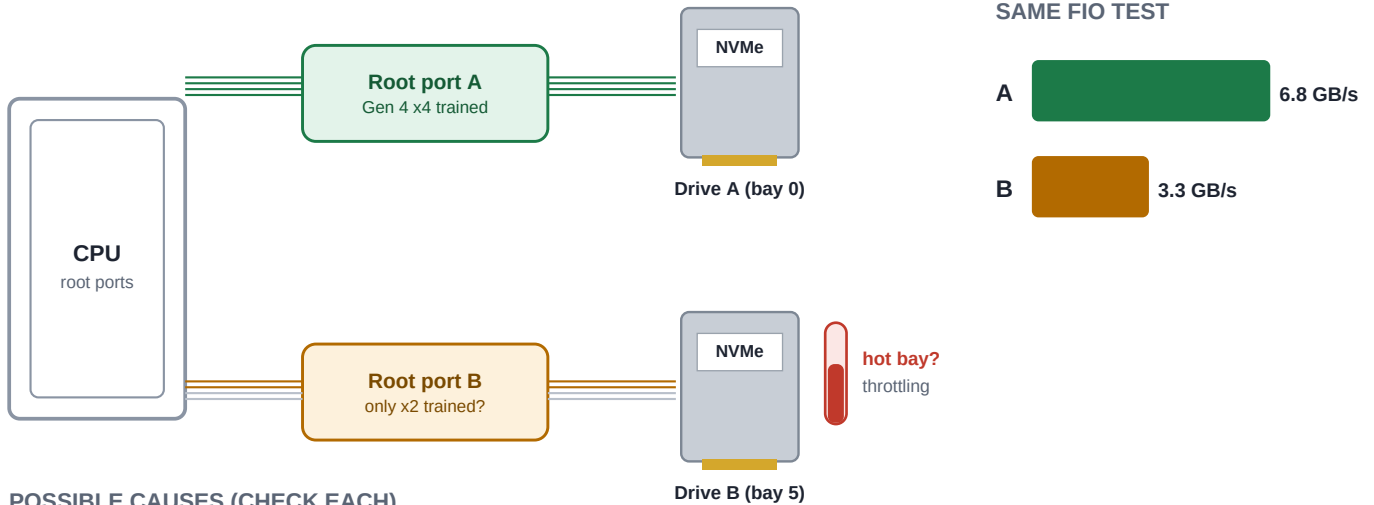
Half speed? Check Link, Path, Heat, Firmware, Format, Test, Health. Then swap bays.

EXPLANATION (understand it)

Two identical drives should give identical results. So the job is to find what is different: the link, the path, the temperature, the firmware, or the test.

Start with checks that change nothing (read `LnkSta`, temperatures, firmware) before you move hardware. Then the bay swap tells you whether to blame the drive or the slot it sits in.

SEE IT: TOPOLOGY / DIAGRAM



POSSIBLE CAUSES (CHECK EACH)

- link x2 / Gen 3
- shared switch uplink
- hot bay: throttling
- different firmware
- drive full / formatted differently
- test not identical
- failing drive

Two identical drives should match. Compare the link each one trained to, the path it takes, its temperature, firmware and format. Then swap bays.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Same test, different results

```
$ sudo fio --name=t --filename=/dev/nvme0n1 --rw=read --bs=128k --iodepth=32 --direct=1 --runtime=60 | grep
READ:
  READ: bw=6812MiB/s (7143MB/s), io=399GiB, run=60001msec
$ sudo fio --name=t --filename=/dev/nvme1n1 --rw=read --bs=128k --iodepth=32 --direct=1 --runtime=60 | grep
READ:
  READ: bw=3321MiB/s (3482MB/s), io=195GiB, run=60001msec
```

→ nvme1 gets about half of nvme0, with the identical command.

Link, firmware and heat

```
$ sudo lspci -vv -s 5e:00.0 | grep LnkSta: ; sudo lspci -vv -s 5f:00.0 | grep LnkSta:
LnkSta: Speed 16GT/s, Width x4
LnkSta: Speed 16GT/s, Width x4
$ sudo nvme list | awk '{print $1, $NF}'
/dev/nvme0n1 GDC5602Q
/dev/nvme1n1 GDC5602Q
$ sudo nvme smart-log /dev/nvme1 | grep -i -E "^temperature|T1 Trans"
temperature          : 79 C (352 Kelvin)
Thermal Management T1 Trans Count : 118
```

→ Both links are Gen 4 x4 and firmware matches, so those are ruled out.

→ 79 C and 118 throttling events: the drive slows itself down to cool off. Check that bay's airflow.

Failure follows the part

Reread: Module 10: Isolation techniques

You move a suspect GPU from Server A into known-good Server B, and Server B now fails the same way. What does this prove, what does it not prove, and what do you do next with both servers and the GPU?

 ANSWER (simple words: write this)

What it proves: the GPU is very likely bad, because the failure moved with it into a system that was working.

What it does not prove: that Server A is healthy. A bad slot or a power problem in Server A could have damaged the GPU in the first place, and it could damage the next good GPU too.

Next steps:

1. Tag the GPU as bad and quarantine it, so nobody reuses it.
2. Put Server B back to its original state (its own good GPU) and retest it, to make sure it still passes.
3. Inspect Server A's slot (bent pins, damage, burn marks) and the GPU power cable, then test Server A with a known-good GPU.
4. Document the serial numbers of both servers and the GPU, the slots, and every test result.
5. Start the RMA for the GPU, with failure analysis (FA) if needed.

REMEMBER IT

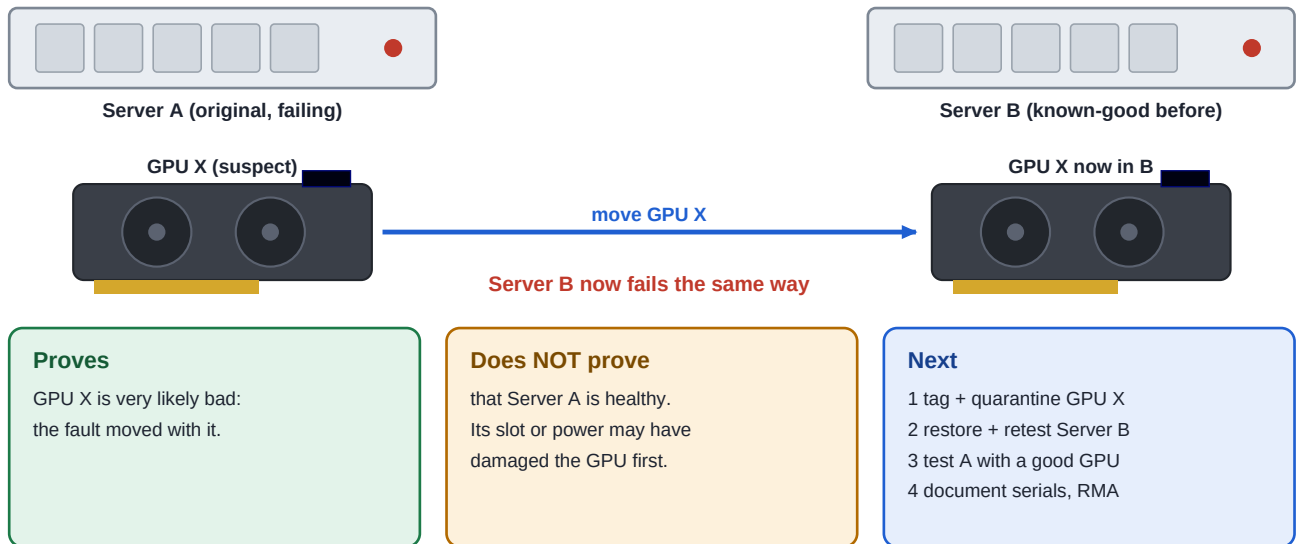
Follows the part = bad part. But it does not prove Server A is OK.

► **EXPLANATION** (understand it)

A swap answers one question: "Is it the part?" It does not answer "Why did the part fail?" If Server A's slot fried the GPU, putting a new GPU into Server A just fries another one.

So you finish the job on both sides: prove Server B is still good, and prove Server A is safe for a new part.

SEE IT: TOPOLOGY / DIAGRAM



When the failure follows the part into a good system, the part is bad. The original system still has to be checked before it gets another good part.

>_ TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

In Server B: the same error, from the same GPU

```
$ nvidia-smi -q | grep -E "Serial Number|Bus Id"
  Serial Number      : 1652322004417
    Bus Id           : 00000000:3D:00.0
$ dmesg -T | grep -i xid
[Fri Oct  2 11:05:43 2026] NVRM: Xid (PCI:0000:3d:00): 79, pid=0, GPU has fallen off the bus.
```

→ Serial ...4417 is the GPU that came from Server A, and it shows the same Xid 79 in Server B.

Record both servers

```
$ sudo dmidecode -s system-serial-number      # on Server A, then Server B
S421A0991
S421A1047
```

→ The repair record needs all three serials: Server A, Server B and the GPU.

ip a shows 169.254

Reread: Module 7: The basics; Reading ip a; DHCP

Here is part of the ip a output from a server that cannot reach anything on the network. What does it tell you, what is most likely wrong, and what would you check next?

```
3: ens2f1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc mq state UP  
    link/ether b8:ce:f6:4a:10:22 brd ff:ff:ff:ff:ff:ff  
    inet 169.254.88.4/16 brd 169.254.255.255 scope link ens2f1
```

ANSWER (simple words: write this)

What it tells me:

- Interface ens2f1 is UP and LOWER_UP, so the NIC is enabled and has a physical link. The cable and switch port pass a link.
- MTU 9000 (jumbo frames). MAC address b8:ce:f6:4a:10:22.
- The address 169.254.88.4/16 with "scope link" is a self-assigned link-local address. There is no "dynamic".

Most likely wrong: DHCP failed. The server asked for an address and no DHCP server answered, so it gave itself one. With a 169.254 address it cannot reach anything beyond its own link, and there is no default gateway.

What I would check next:

1. The switch port VLAN: if the port is on the wrong VLAN, there is no DHCP server there to answer.
2. The DHCP server: is it up, and does it have a free address or a reservation for this MAC?
3. The DHCP relay on the router, if the DHCP server is on another subnet.
4. The cable is in the right port. On a dual-port card the other port may be the one that is configured.
5. MTU 9000 matches the switch.
6. Then renew (for example `sudo dhclient -v ens2f1`, or restart the connection in NetworkManager) and confirm a real address with "dynamic", a default route in ip route, and DNS. Document the fix.

REMEMBER IT

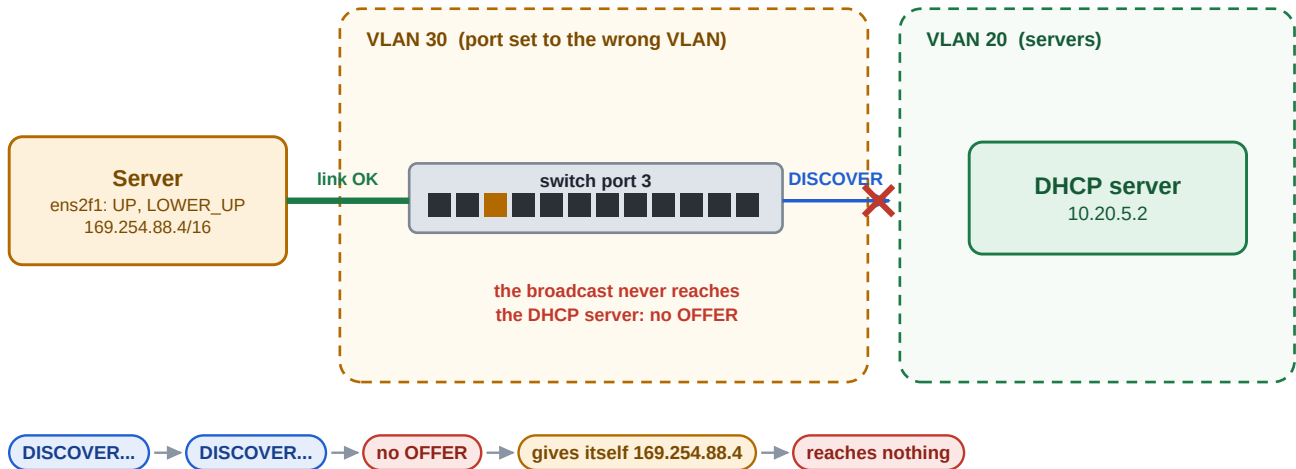
169.254 = "I asked DHCP and nobody answered." The link is fine; the path to DHCP is not.

► EXPLANATION (understand it)

Read it from the bottom of the stack up: the link is good (LOWER_UP), so the cable and NIC are not the problem. The address is the clue: 169.254.x.x only appears when DHCP got no answer.

So the fault is between the server and the DHCP server: usually the VLAN on the switch port, a down DHCP server, or a missing relay.

SEE IT: TOPOLOGY / DIAGRAM



Also check: DHCP server up, relay on the router, cable in the right port, MTU 9000 matches the switch.

169.254.x.x means "I asked DHCP and nobody answered". The link is up, so look at what stands between the server and the DHCP server.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

No gateway, and DHCP gets no offers

```
$ ip route
169.254.0.0/16 dev ens2f1 proto kernel scope link src 169.254.88.4
(no "default via ..." line: no gateway)
$ sudo dhclient -v ens2f1
DHCPDISCOVER on ens2f1 to 255.255.255.255 port 67 interval 4
DHCPDISCOVER on ens2f1 to 255.255.255.255 port 67 interval 9
No DHCPOFFERS received.
```

→ The server keeps asking; nobody answers. Look at the VLAN and the DHCP server next.

The link itself is fine

```
$ ethtool ens2f1 | grep -E "Speed|Link detected"
Speed: 25000Mb/s
Link detected: yes
```

→ Physical layer OK, so do not swap the cable or the card yet.

Memory: DIMM, board, CPU, or configuration

Reread: Module 5: Is it the DIMM or the system?

After a memory upgrade, a server shows less memory than installed and logs corrected errors on two different channels. How would you decide whether the cause is the new DIMMs, the slots or motherboard, the CPU, or how the memory was installed and configured?

 ANSWER (simple words: write this)

1. Record the evidence: `free -h` (how much is missing), `dmidecode -t memory` (which slots are filled, sizes, serials, "No Module Installed"), and the errors from the BMC SEL and `ras-mc-ctl --summary` (which slots and channels).
2. Check the installation and configuration first (most likely after an upgrade):
 - Every new DIMM fully seated with both latches closed.
 - The slots follow the population rules in the service guide.
 - The new DIMMs match the old ones (same type: RDIMM, DDR5; supported part; same speed and size where required).
 - The BIOS is current enough to support the new DIMMs, no DIMMs are disabled in the BIOS, and the memory mode is right (mirroring or sparing hide memory).
3. Look at the pattern:
 - Errors only on the new DIMMs (on both channels) points to the new DIMMs or how they were installed.
 - Errors on many channels of one CPU points to the CPU (seating, bent socket pins, memory controller).
4. Swap test, one change at a time: move a suspect new DIMM to a slot on another channel, and put a known-good DIMM in its slot. Clear the logs and run a memory test.
 - Errors follow the DIMM: the DIMM is bad.
 - Errors stay with the slot: the slot or motherboard.
5. If it points to the CPU: reseal it and check the socket pins, then try a known-good CPU.
6. Verify the full amount shows, run a stress test, and document slots, serials and results.

REMEMBER IT

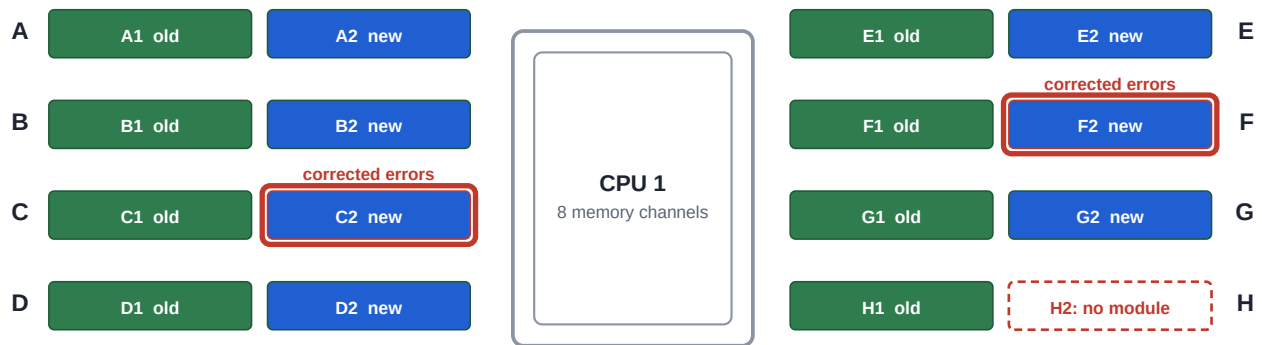
Map it first: which slots, which channels, which DIMMs are new. Then: Install, DIMM, Slot, CPU.

► **EXPLANATION** (understand it)

After an upgrade, start with what changed: the new DIMMs and how they went in. Most problems are a DIMM not fully latched, a DIMM in the wrong slot, or parts that should not be mixed.

The trick is to turn "errors on two channels" into a map. If both channels with errors are exactly the slots that got new DIMMs, suspect those DIMMs. If the errors stay with a slot after a swap, suspect the board. If they spread across one CPU, suspect that CPU.

SEE IT: TOPOLOGY / DIAGRAM



New DIMMs errors follow them when moved to another channel

Slots / board errors stay in the slot with a known-good DIMM

CPU errors on many channels of one CPU: reseal, check pins

Install / config wrong slots, mixed parts, not latched, BIOS, mirroring

Map the errors to slots and channels first. Two channels with new DIMMs and errors, plus a missing module, point you at the new parts and how they were installed.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Less memory: one new DIMM is not seen

```
$ free -h | head -2
      total        used        free      shared  buff/cache   available
Mem:    944Gi       61Gi       871Gi        2.1Gi        14Gi       877Gi
$ sudo dmidecode -t memory | grep -E "Locator: DIMM_[GH]2|Size" | tail -4
Size: 64 GB
Locator: DIMM_G2
Size: No Module Installed
Locator: DIMM_H2
```

→ 16 x 64 GB were installed, but H2 is not detected: one DIMM missing. Reseat it (latches!) before anything else.

Corrected errors on two channels

```
$ sudo ras-mc-ctl --summary
Memory controller events summary:
  Corrected on DIMM Label(s): 'CPU_SrcID#0_MC#1_Chan#0_DIMM#1' location: 1:0:1:-1 errors: 212
  Corrected on DIMM Label(s): 'CPU_SrcID#0_MC#2_Chan#1_DIMM#1' location: 2:1:1:-1 errors: 87
```

→ Both errors are on DIMM#1 (the second slot of a channel), which is where the new DIMMs went. Suspect the new DIMMs or their installation, then prove it with a swap.

Replacement did not fix it

Reread: Module 10: When a replacement does not fix it

A GPU kept failing its stress test, so it was replaced. The new GPU fails the same test, the same way, in the same slot. What does this suggest, and what should happen next instead of replacing another part?

 **ANSWER** (simple words: write this)

What it suggests: the GPU was probably never the cause. Two different GPUs failing the same way in the same slot point to something that slot shares:

- the slot itself, the riser or the PCIe cable to it,
- the GPU's auxiliary power cable or the PSU feeding it,
- a retimer on that path,
- cooling for that slot (airflow, a missing baffle or blank, a failed fan),
- BIOS, firmware or driver settings,
- or the test itself.

The first GPU may be good, so do not send it for RMA yet.

What should happen next (instead of another part):

1. Stop replacing parts and collect data: the exact error (for example Xid 79, AER errors), temperatures and power during the test, the BMC SEL, and firmware and driver versions.
2. Compare with a golden unit running the same test.
3. Test the slot: put a known-good GPU from a passing server into this slot, and put this GPU into a known-good slot. One change at a time.
4. Inspect the slot path: riser seating, cables, power cable, airflow parts.
5. Retest the first GPU in a good system; if it passes, return it to stock.
6. If still unclear, escalate with the data. Document everything, including that the replacement did not fix it.

REMEMBER IT

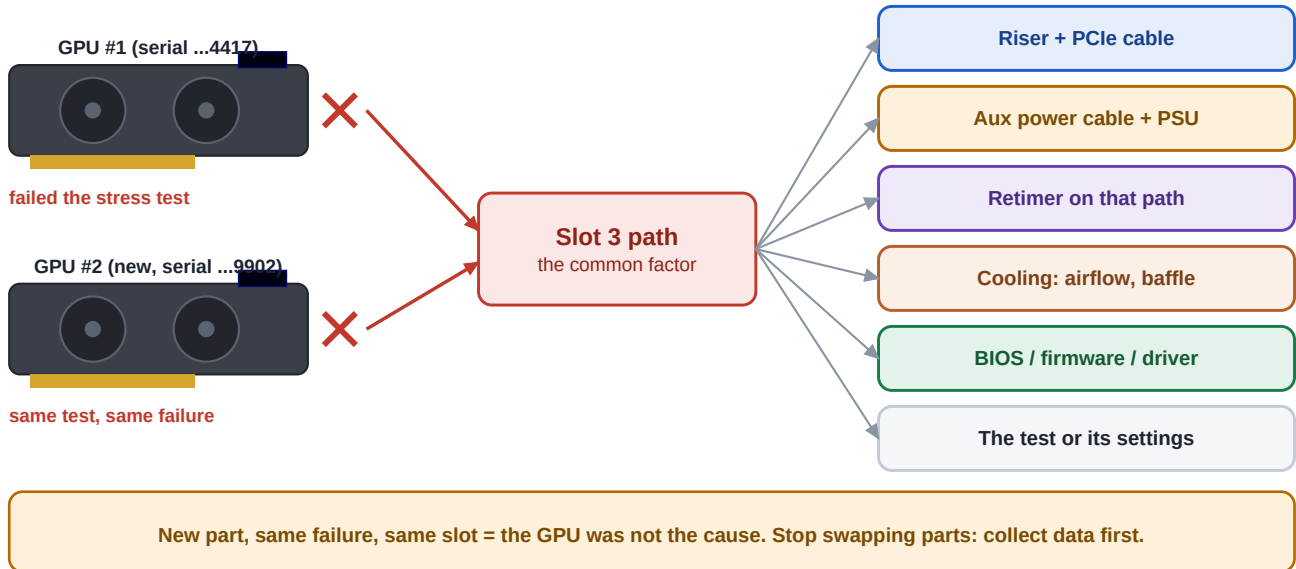
New part, same failure, same slot = not the part. Stop swapping; start measuring.

► **EXPLANATION** (understand it)

Replacing parts until the problem goes away is called "shotgunning" or using the "parts cannon". It wastes good parts and often never finds the real cause.

When a brand-new part fails exactly like the old one, the evidence is telling you the problem is around the part, not in it. That is the moment to stop and measure.

SEE IT: TOPOLOGY / DIAGRAM



Two different GPUs failing the same way in the same slot point away from the GPU and towards everything that slot shares.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Two GPUs, same slot, same error

```
$ journalctl -k | grep -i xid
Oct 01 14:22:10 srv-r07-u12 kernel: NVRM: Xid (PCI:0000:3d:00): 79, pid=0, GPU has fallen off the bus.
Oct 02 11:05:43 srv-r07-u12 kernel: NVRM: Xid (PCI:0000:3d:00): 79, pid=0, GPU has fallen off the bus.
$ nvidia-smi -q -i 3 | grep "Serial Number"          # before and after the swap
  Serial Number                                     : 1652322004417
  Serial Number                                     : 1652322009902
```

→ Same PCI address (the same slot) both days, but two different serial numbers. The common factor is the slot.

A clue from the BMC

```
$ sudo ipmitool sensor | grep -i -E "GPU[0-9] Temp"
GPU2 Temp      | 66.000      | degrees C | ok
GPU3 Temp      | 91.000      | degrees C | cr
GPU4 Temp      | 64.000      | degrees C | ok
```

→ Slot 3 runs 25 degrees hotter than its neighbors: check that slot's airflow (baffle, blank, fan) before touching the GPU again.

Filtering large logs

Reread: Module 2: Finding one event in a large log

The journal on a busy server has hundreds of thousands of lines. Write commands to (a) show only error-level messages from the current boot, (b) show everything logged from 9:00 to 9:15 this morning, and (c) find which error message repeats most often. Explain each part.

 **ANSWER** (simple words: write this)

(a) Errors from the current boot only:

```
journalctl -b -p err
```

-b = this boot only. -p err = priority "err" and worse (crit, alert, emerg). Add -k for kernel messages only.

(b) Everything from 9:00 to 9:15 this morning:

```
journalctl --since "09:00" --until "09:15"
```

A time with no date means today. This lines the logs up with the time the problem happened.

(c) Which error repeats most often:

```
journalctl -b -p err -o cat | sort | uniq -c | sort -rn | head
```

- -o cat prints only the message text, without the time and host, so identical messages look identical.
- sort puts identical lines next to each other.
- uniq -c counts each group of identical lines.
- sort -rn sorts by that count, biggest first (r = reverse, n = numeric).
- head shows the top 10.

To save the result as evidence, add > errors_top.txt.

REMEMBER IT

-b this boot, -p err errors, --since/--until a time window, sort | uniq -c | sort -rn top repeats.

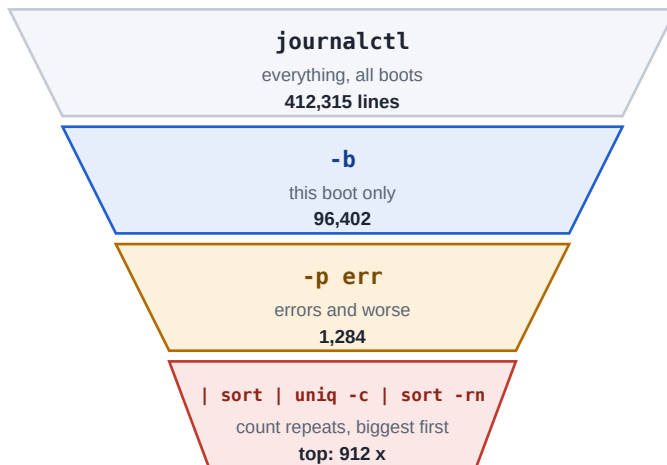
► **EXPLANATION** (understand it)

Each option throws away lines you do not need. Start wide, then narrow: this boot, then errors only, then count. A time window is the other way to narrow: if users say "it broke at 9:05", look at 9:00 to 9:15.

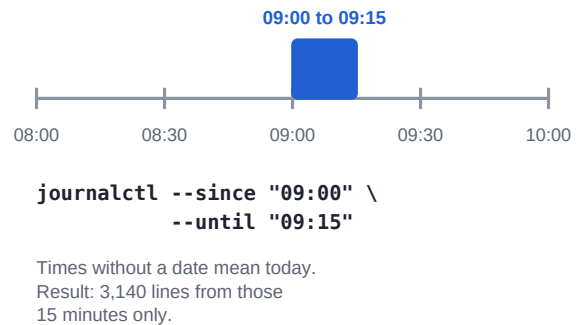
The pipe trick in (c) works with any command that prints lines, not just journalctl.

SEE IT: TOPOLOGY / DIAGRAM

(A) AND (C): NARROW IT DOWN



(B): A TIME WINDOW



Each option throws away lines you do not need: this boot, then errors only, then count the repeats. A time window lines the logs up with when the problem happened.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

(a) Errors from this boot

```
$ journalctl -b -p err
Oct 02 08:05:51 srv-r07-u12 kernel: nvme nvme1: controller is down; will reset: CSTS=0xffffffff
Oct 02 09:07:13 srv-r07-u12 systemd[1]: data-sync.service: Failed with result 'exit-code'.
Oct 02 09:07:43 srv-r07-u12 systemd[1]: data-sync.service: Failed with result 'exit-code'.
```

→ Only error-level lines from the current boot.

(b) A 15-minute window

```
$ journalctl --since "09:00" --until "09:15"
Oct 02 09:00:01 srv-r07-u12 CROND[44120]: (root) CMD (/usr/local/bin/backup.sh)
Oct 02 09:06:58 srv-r07-u12 kernel: nvme nvme1: I/O 412 QID 9 timeout, aborting
Oct 02 09:07:13 srv-r07-u12 systemd[1]: data-sync.service: Failed with result 'exit-code'.
```

→ Everything at every level, but only from 09:00 to 09:15. Here a backup starts, the NVMe drive times out, and the sync job fails.

(c) The most repeated error

```
$ journalctl -b -p err -o cat | sort | uniq -c | sort -rn | head -3
912 data-sync.service: Failed with result 'exit-code'.
36 nvme nvme1: controller is down; will reset: CSTS=0xffffffff
4 mce: [Hardware Error]: Machine check events logged
```

→ The first column is the count. The sync job failed 912 times, probably because of the NVMe drive below it.

Network card checks by command

Reread: Module 3: The four layers; Module 7: Reading ip a

A new network card was installed. Write the Linux commands you would run, in order, to confirm the system found it, a driver is attached, it has a link, and it has an address. Say what each output tells you.

 **ANSWER** (simple words: write this)

1. Found it? `lspci | grep -i ethernet`

Shows the card on the PCIe bus, with its address (for example 3b:00.0 and 3b:00.1 for two ports). If it is not here, it is a physical, BIOS or slot problem.

2. Driver attached? `lspci -k -s 3b:00.0` and `dmesg | grep -i mlx5`

`lspci -k` must show "Kernel driver in use: mlx5_core". `dmesg` shows the driver loading, the firmware version and the interface name. No driver line = a driver problem, not a bad card.

3. Link? `ip link show ens3f0` and `ethtool ens3f0`

`ip link` shows UP and LOWER_UP (link) or NO-CARRIER (no link). `ethtool` shows "Link detected: yes" and the speed (for example 100000Mb/s). No link = cable, transceiver, dirty fiber or switch port.

4. Address? `ip a show ens3f0` and `ip route`

`ip a` shows the inet line with the IP and mask, and "dynamic" if it came from DHCP (169.254 = DHCP failed). `ip route` shows the default gateway.

Finally: `ping -c 3 10.20.5.1` (the gateway) to prove it can talk.

REMEMBER IT

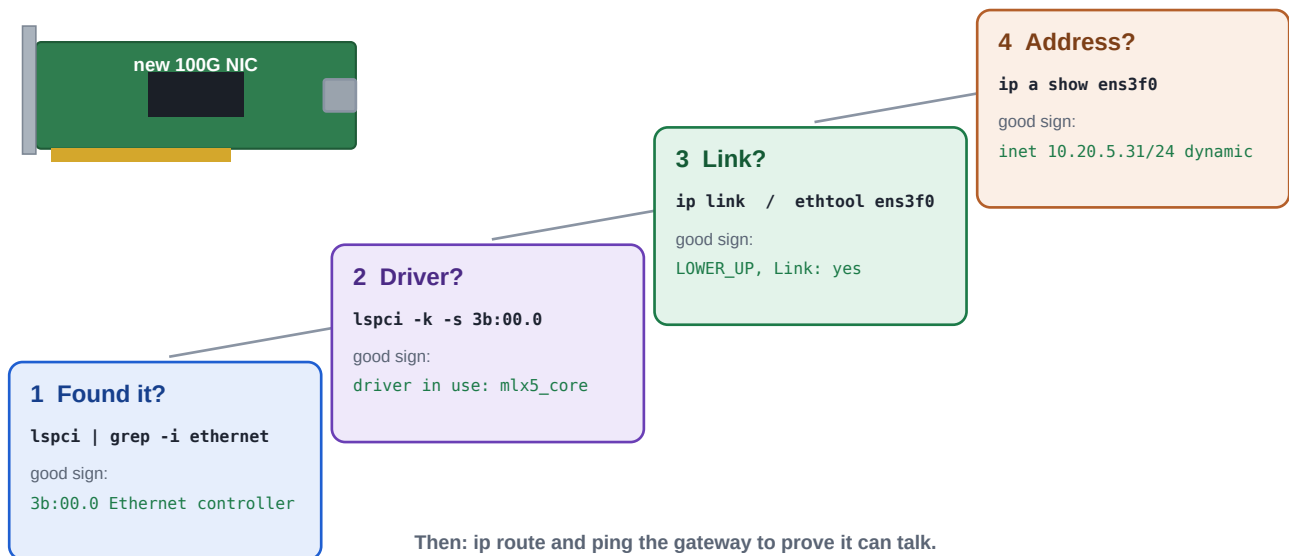
Four steps up: Found (`lspci`), Driver (`lspci -k`), Link (`ethtool`), Address (`ip a`).

► **EXPLANATION** (understand it)

This is the four-layer check done with commands: bus, driver, link, address. Go in order, because each step depends on the one before. A card with no driver can never have a link, and a port with no link can never get an address.

Write the exact command and say what a good result looks like. That is what "command questions" are graded on.

SEE IT: TOPOLOGY / DIAGRAM



Climb one step at a time: the bus sees it, a driver owns it, the cable gives a link, and the network gives an address.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Steps 1 and 2: found, with a driver

```
$ lspci | grep -i ethernet
3b:00.0 Ethernet controller: Mellanox Technologies MT2892 Family [ConnectX-6 Dx]
3b:00.1 Ethernet controller: Mellanox Technologies MT2892 Family [ConnectX-6 Dx]
$ lspci -k -s 3b:00.0
3b:00.0 Ethernet controller: Mellanox Technologies MT2892 Family [ConnectX-6 Dx]
Subsystem: Mellanox Technologies Device 0016
Kernel driver in use: mlx5_core
$ dmesg | grep -i mlx5 | tail -2
[ 8.402117] mlx5_core 0000:3b:00.0: firmware version: 22.36.1010
[ 9.115230] mlx5_core 0000:3b:00.0 ens3f0: renamed from eth0
```

→ Two ports (.0 and .1). The driver is mlx5_core, and the first port became ens3f0.

Steps 3 and 4: link and address

```
$ ethtool ens3f0 | grep -E "Speed|Link detected"
Speed: 100000Mb/s
Link detected: yes
$ ip a show ens3f0 | grep inet
inet 10.20.5.31/24 brd 10.20.5.255 scope global dynamic ens3f0
$ ip route | grep default
default via 10.20.5.1 dev ens3f0 proto dhcp
```

→ 100G link up, a DHCP address and a default gateway. Last check: ping the gateway.

First commands on an unknown server

Reread: Module 2; Module 3; Command Quick Reference

You log in to a server you have never seen before to look for a hardware problem. Write the commands you would run in your first few minutes, in a sensible order, and what each one tells you.

 **ANSWER** (simple words: write this)

1. Where am I?
 - `hostname` and `cat /etc/os-release`: which server and which Linux version.
 - `uptime`: how long since the last reboot, and the load. A recent reboot may mean a crash.
2. What hardware does it have?
 - `lscpu`: CPUs, sockets and cores.
 - `free -h`: total memory (is it what it should be?).
 - `lsblk` and `nvme list`: drives and their sizes.
 - `lspci` (and `lspci -k`): every PCIe device and its driver. Compare with the build sheet.
3. What is going wrong now?
 - `dmesg -T --level=err,warn | tail -50`: recent kernel errors and warnings, with readable times.
 - `journalctl -p err -b`: errors from this boot.
 - `journalctl --list-boots` and `journalctl -b -1`: earlier boots, and the boot before a crash.
4. What did the hardware log?
 - `sudo ipmitool sel elist`: the BMC event log (memory, PCIe, power, temperature events).
 - `sudo ipmitool sensor`: temperatures, fans, PSUs and voltages right now.
5. How is it connected?
 - `ip a` and `ip route`: interfaces, addresses and the gateway.

Save what I find to files (for example `dmesg -T > unit123_dmesg.txt`) and change nothing until I understand it.

REMEMBER IT

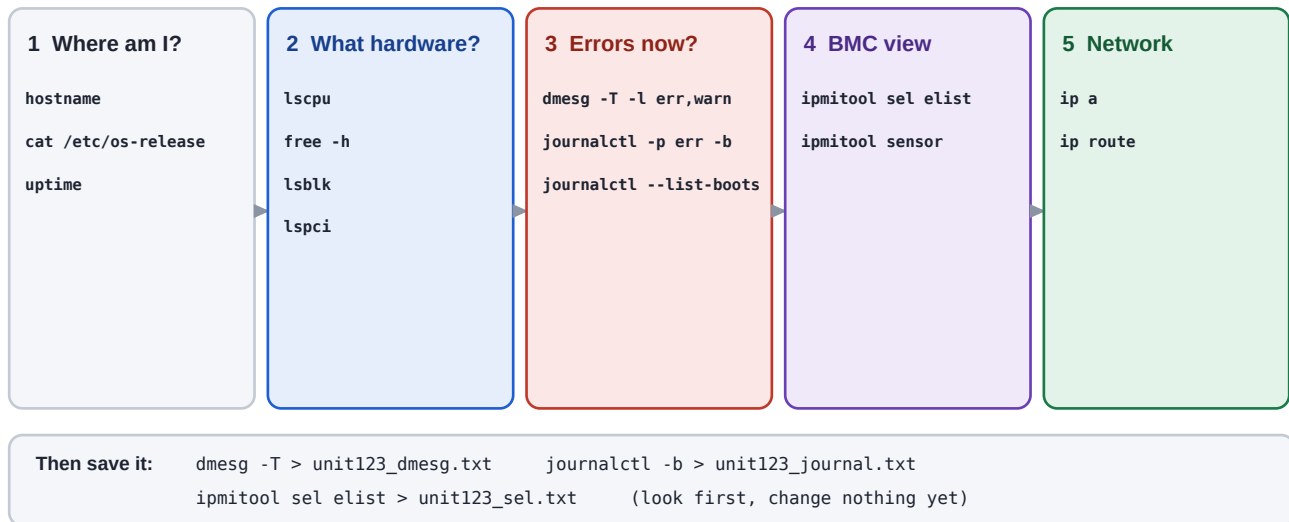
Who am I on? What hardware? What is wrong now? What did the BMC see? How is it connected?
Look before you touch.

► **EXPLANATION** (understand it)

The order goes from general to specific: identify the machine, list what is in it, then look for errors in the OS logs and in the BMC. Reading changes nothing, so it is always safe to do first.

On an unknown server, compare what you see with what should be there: missing memory, a missing card or a missing drive is often the whole answer.

SEE IT: TOPOLOGY / DIAGRAM



A sensible order: who and what the server is, what hardware it has, what is going wrong now, what the BMC recorded, and how it is connected.

>_ TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Who and what

```
$ hostname; grep PRETTY /etc/os-release; uptime
srv-r07-u12
PRETTY_NAME="Rocky Linux 9.4 (Blue Onyx)"
09:14:02 up 0 days, 0:41, 1 user, load average: 0.42, 0.51, 0.48
$ lscpu | grep -E "^Socket|^CPU\s\>"; free -h | grep Mem
CPU(s):                128
Socket(s):              2
Mem:                   944Gi       12Gi       925Gi       1.1Gi       7.2Gi       931Gi
```

→ Up only 41 minutes: it rebooted recently (crash?). 944 GiB looks short for a 1 TB build: one more clue.

Errors in the OS and in the BMC

```
$ dmesg -T --level=err,warn | tail -2
[Fri Oct 2 08:33:12 2026] EDAC MC1: 87 CE memory read error on CPU_SrcID#0_MC#1_Chan#0_DIMM#1
[Fri Oct 2 08:33:40 2026] mce: [Hardware Error]: Machine check events logged
$ sudo ipmitool sel elist | tail -2
71 | 10/02/2026 | 08:31:55 | Memory DIMM_E2 | Correctable ECC | Asserted
72 | 10/02/2026 | 08:31:56 | System Event | OEM System boot event | Asserted
```

→ Memory errors in Linux and in the BMC, plus a reboot at 08:31: start with memory (Module 5).

Slow server: processes and load

Reread: Module 2: Processes and system load

Users report that a server has become slow. Write the commands you would use to find which processes use the most CPU and memory, compare the load with the number of cores, and tell whether CPU, memory, or storage is the bottleneck. Explain how to read the key numbers.

 **ANSWER** (simple words: write this)

Find the busy processes:

- `top` (press P to sort by CPU, M by memory): live view of the busiest processes.
- `ps aux --sort=-%cpu | head` and `ps aux --sort=-%mem | head`: the top processes by CPU and by memory.

Compare load with cores:

- `uptime` shows the load average for 1, 5 and 15 minutes. `nproc` shows the number of CPU cores.
- If the load is higher than the number of cores, work is waiting for CPU. Load 62 on 32 cores means about twice as much work as the CPUs can do.

Find the bottleneck:

- CPU: in `top`, `%us` (user) and `%sy` (system) are high and `%id` (idle) is near 0, while `%wa` is low.
- Memory: `free -h` shows "available" near zero, swap is being used, and `vmstat 1` shows `si/so` (swap in/out) above 0 every second. The server is swapping, which is very slow.
- Storage: in `top`, `%wa` (iowait) is high: the CPU is waiting for disk. `iostat -x 1` shows a disk near 100 %util with high await (milliseconds per request).

Then look at the process using that resource, and compare with normal behavior or a golden unit before changing anything.

REMEMBER IT

Compare load average to cores. Then look at `%us`, `%wa`, available, swap. Busy CPU, short memory, or waiting disk?

► **EXPLANATION** (understand it)

Load average is the number of tasks running or waiting. Compare it to the number of cores: on 32 cores, a load of 32 means "fully busy", 64 means "a queue as long as the work".

The three key questions: are the CPUs busy (`%us`)? Is memory full and swapping (available, `si/so`)? Is the CPU waiting for disks (`%wa`, `%util`)? Only one of them is usually the main bottleneck.

SEE IT: TOPOLOGY / DIAGRAM

Who is using it?

top

ps aux --sort=-%cpu | head

ps aux --sort=-%mem | head

CPU

uptime (load average)
nproc (number of cores)
top (%us %sy)

It is the bottleneck when:

load average above the
number of cores, %us high,
%wa low

Memory

free -h (available)
vmstat 1 (si so)
top (MiB Swap used)

It is the bottleneck when:

"available" near zero,
swap in / out (si, so)
busy every second

Storage

top (%wa = iowait)
iostat -x 1
(%util, await)

It is the bottleneck when:

%wa high, a disk at
~100 %util, await of
many milliseconds

Find the busy processes, then decide which resource is full. Compare load with the number of cores, check available memory and swap, and check iowait and disk %util.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Load versus cores

```
$ uptime
14:02:11 up 41 days, 3:12, 2 users, load average: 62.14, 58.90, 41.22
$ nproc
32
```

→ Load 62 on 32 cores: about twice as much work as the CPUs can handle.

top: who is using it, and which resource

```
$ top -b -n 1 | head -9
top - 14:02:12 up 41 days, 3:12, 2 users, load average: 62.14, 58.90, 41.22
Tasks: 612 total, 59 running, 553 sleeping, 0 stopped, 0 zombie
%Cpu(s): 95.1 us, 3.2 sy, 0.0 ni, 1.2 id, 0.3 wa, 0.0 hi, 0.2 si, 0.0 st
MiB Mem : 257612.4 total, 41250.1 free, 190112.7 used, 26249.6 buff/cache
MiB Swap: 8192.0 total, 8192.0 free, 0.0 used. 64188.3 avail Mem

   PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM     TIME+ COMMAND
 48211 app       20   0   92.1g   61.3g   12.1g R   2890   24.3   812:44.10 java
 50112 etl      20   0   12.4g    8.1g    1.2g R    201    3.2    41:02.33 python3
```

- 95% us, 1% idle, 0.3% wa: the CPUs are full; disks are not the problem.
- Swap 0 used, 64 GB available: memory is fine.
- One java process uses 2890% CPU (about 29 cores). Bottleneck: CPU, caused by that process.

If it were storage instead

```
$ iostat -x 1 1 | grep -E "Device|nvme0n1"
Device  r/s    w/s   rkB/s   wkB/s  r_await w_await  aqu-sz  %util
nvme0n1 412.0  9120.0 52736.0 1167360.0 1.21   48.30   441.2  99.8
```

- A disk at 99.8% util with 48 ms per write: storage is the bottleneck in that case (and top would show high %wa).

Set B

Questions 21 to 40

Part 1, Foundations: questions 21 to 29. Part 2, Troubleshooting: questions 30 to 36. Part 3, Linux in practice: questions 37 to 40.

What Linux is and why servers use it

Reread: Module 2: What Linux is and why data centers use it

In your own words, what is Linux? Name two Linux distributions used on servers, and give four specific reasons Linux runs most data center servers.

 **ANSWER** (simple words: write this)

Linux is an operating system. Strictly, "Linux" is the kernel: the core part that talks to the hardware. It is packaged with tools into distributions. It is open source: the code is public and free to use.

Two server distributions: Red Hat Enterprise Linux (RHEL) and Ubuntu Server. (Also Rocky Linux and SUSE.)

Four reasons it runs most data center servers:

1. Stable: servers run for months without a reboot.
2. Low cost: most distributions are free; paid ones like RHEL charge for support, not the software.
3. Built for remote work: everything is done by command line and scripts over the network, with no screen. One person can manage hundreds of servers.
4. Transparent about hardware: tools like `lspci` and `dmesg` show exactly what the hardware is doing, which is what diagnostics need.

Also: secure, efficient, frequently updated, and most AI and cloud software is built for Linux first.

REMEMBER IT

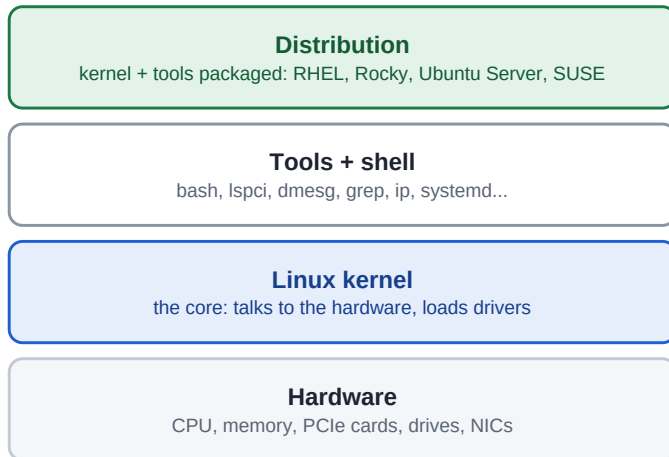
Linux = kernel; distro = kernel + tools. Reasons: S-L-R-T ("Servers Love Running Tux").

► **EXPLANATION** (understand it)

Think of the kernel as a car's engine, and a distribution as the whole car built around it: same engine, different models (RHEL, Rocky, Ubuntu, SUSE). In a data center the "car" has no dashboard screen: you drive it by typing commands over the network.

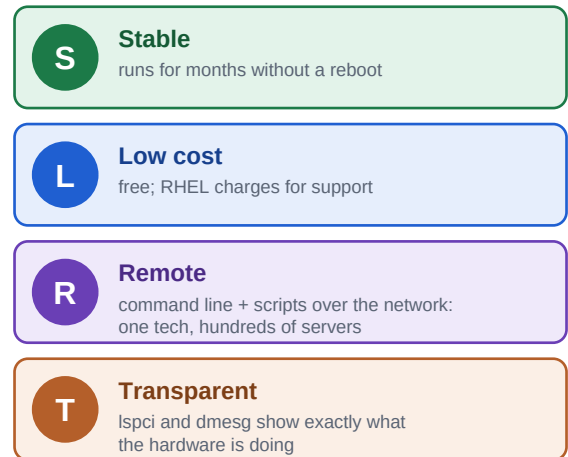
SEE IT: TOPOLOGY / DIAGRAM

WHAT "LINUX" MEANS



Strictly, "Linux" = the kernel. A distro = kernel + tools.

WHY DATA CENTERS RUN IT: S-L-R-T



Bonus: secure, efficient, AI and cloud software built for Linux first

The Linux stack (left) and the four reasons to memorize (right). Mnemonic: "Servers Love Running Tux" = Stable, Low cost, Remote, Transparent.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Which Linux, which kernel, how long running

```
$ cat /etc/os-release | head -2
NAME="Rocky Linux"
VERSION="9.4 (Blue Onyx)"
$ uname -r
5.14.0-427.13.1.el9_4.x86_64
$ uptime
09:14:02 up 182 days,  4:11,  1 user,  load average: 3.12, 3.40, 3.38
```

- The distribution is Rocky Linux 9.4; uname -r shows the kernel version.
- 182 days without a reboot: "stable".

Reading ip a output

Reread: Module 7: Reading ip a

Here is part of the ip a output from a server. List every piece of information you can read from it. Then name two things this command does not show, and where you would find them.

```
2: ens1f0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc mq state UP  
    link/ether 3c:ec:ef:12:34:56 brd ff:ff:ff:ff:ff:ff  
    inet 10.20.5.14/22 brd 10.20.7.255 scope global dynamic ens1f0
```

ANSWER (simple words: write this)

What I can read:

- Interface number 2, named ens1f0 (Ethernet, slot 1, port 0). The name can change if the card moves slots.
- UP: the interface is enabled. LOWER_UP: it has a physical link (cable and switch port work). BROADCAST, MULTICAST: it can send those types of traffic.
- mtu 9000: jumbo frames. The switch port must use the same MTU.
- qdisc mq: the queue setup (multi-queue); state UP: the link is up.
- link/ether 3c:ec:ef:12:34:56: the MAC address. brd ff:ff:ff:ff:ff:ff: the broadcast MAC.
- inet 10.20.5.14/22: the IPv4 address and a /22 mask (255.255.252.0), so the subnet is 10.20.4.0 to 10.20.7.255.
- brd 10.20.7.255: the broadcast address of that subnet.
- scope global: a normal, routable address.
- dynamic: it came from DHCP (a static address shows "forever" for the lease).

Two things it does not show:

- The default gateway: use ip route.
- The DNS servers: look in /etc/resolv.conf.

(Also the link speed: ethtool ens1f0.)

REMEMBER IT

Read left to right: Name, Flags, MTU, MAC, IP/mask, brd, dynamic. Not shown: Gateway, DNS, Speed.

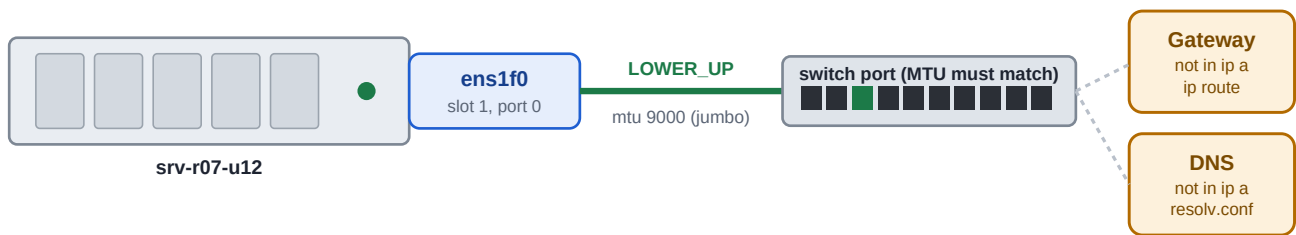
► EXPLANATION (understand it)

ip a answers "who am I on the network?": my name, my MAC, my IP and mask, and whether my link is up. It does not answer "where do I send traffic?" (the gateway, in ip route) or "who turns names into IPs?" (DNS, in resolv.conf).

The /22 is a bigger subnet than the usual /24: four blocks of 256 addresses, 10.20.4.x to 10.20.7.x.

SEE IT: TOPOLOGY / DIAGRAM

WHAT IP A TELLS YOU ABOUT ENS1F0 (AND WHAT IT DOES NOT)



UP / LOWER_UP	enabled / physical link present	inet 10.20.5.14/22	IPv4 + mask 255.255.252.0
mtu 9000	jumbo frames; switch must match	brd 10.20.7.255	broadcast: subnet 10.20.4.0 - 10.20.7.255
link/ether 3c:ec:ef:12:34:56	MAC address	dynamic	came from DHCP (static = "forever")

Also not in ip a: link speed -> **ethtool ens1f0**

Read ip a left to right: name, flags, MTU, MAC, IP/mask, broadcast, dynamic. Gateway, DNS and speed live elsewhere.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

The things ip a does not show

```
$ ip route
default via 10.20.4.1 dev ens1f0 proto dhcp src 10.20.5.14 metric 100
10.20.4.0/22 dev ens1f0 proto kernel scope link src 10.20.5.14 metric 100
$ cat /etc/resolv.conf
search lab.local
nameserver 10.20.5.5
$ ethtool ens1f0 | grep -E "Speed|Link detected"
    Speed: 25000Mb/s
    Link detected: yes
```

→ Gateway from ip route, DNS from resolv.conf, speed from ethtool.

How a name becomes an IP address

Reread: Module 7: DNS

Explain how a Linux server turns the name db01 into an IP address. Include which files are involved, the order they are checked, and what DNS records and caching are.

 **ANSWER** (simple words: write this)

1. Linux checks `/etc/hosts` first. It is a local file that maps names to IP addresses. If `db01` is listed there, Linux uses that and stops. (The order "files, then DNS" is set in `/etc/nsswitch.conf`.)
2. If `db01` is not in `/etc/hosts`, Linux asks the DNS server listed in `/etc/resolv.conf` (the "nameserver" line). If I type a short name, the "search" domain is added, for example `db01.lab.local`.
3. The DNS server answers from its own records if it is in charge of that domain. If not, it asks other DNS servers and passes the answer back.

DNS records are the entries stored on the DNS server. An A record maps a name to an IPv4 address (`db01` to `10.4.2.15`). Others: AAAA (name to IPv6) and CNAME (an alias, another name for the same host).

Caching means the answer is saved for a while (its TTL, time to live), so the next lookup is fast and does not have to go out again. It also means that after a record changes, the old answer can stay until the cache expires.

To test: `nslookup db01` or `dig db01` (ask DNS directly), and `getent hosts db01` (what the system really uses, including `/etc/hosts`).

REMEMBER IT

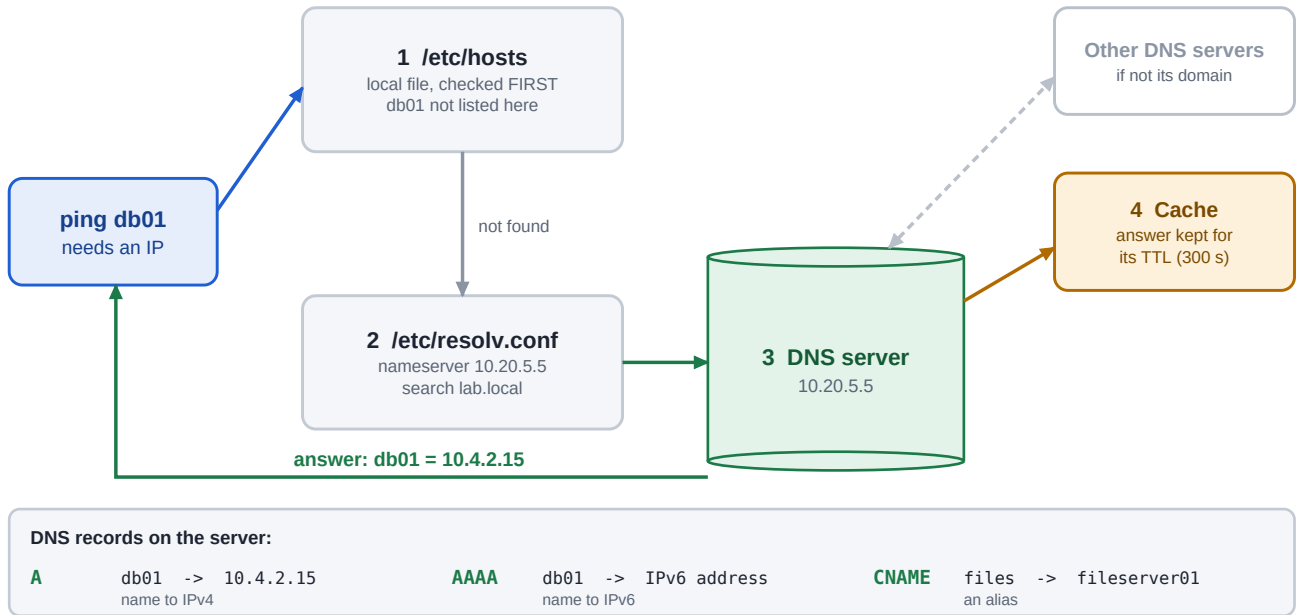
hosts first, then `resolv.conf`, then the DNS server, then cache. A record = name to IPv4.

► **EXPLANATION** (understand it)

It is like finding a phone number: first your own contact list (`/etc/hosts`), then directory assistance (the DNS server from `resolv.conf`). Directory assistance may phone another directory if it does not know, and you remember the answer for a while (cache).

This is why `/etc/hosts` can "win" over DNS: a wrong line in hosts gives a wrong answer even when DNS is perfect.

SEE IT: TOPOLOGY / DIAGRAM



Linux looks in `/etc/hosts` first, then asks the DNS server named in `/etc/resolv.conf`. The DNS server answers from its records (an A record maps a name to an IPv4 address) and the answer is cached.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

The two files, in order

```
$ cat /etc/hosts
127.0.0.1    localhost localhost.localdomain
::1         localhost localhost.localdomain
(no db01 line, so DNS is next)
$ cat /etc/resolv.conf
search lab.local
nameserver 10.20.5.5
```

→ db01 is not in the hosts file, so Linux asks 10.20.5.5.

The DNS answer: an A record with a TTL

```
$ dig db01.lab.local +noall +answer
db01.lab.local.      300    IN      A        10.4.2.15
$ getent hosts db01
10.4.2.15           db01.lab.local
```

→ A = name to IPv4. 300 = the TTL: the answer may be cached for 300 seconds.

PCIe bandwidth and the math

Reread: Module 4: Lanes, generations, and bandwidth

What determines the bandwidth of a PCIe link? List the factors. Then estimate the bandwidth in each direction of a Gen 4 x8 link and show your math.

 **ANSWER** (simple words: write this)

Factors:

1. The generation: the speed per lane. Gen 3 = 8 GT/s, Gen 4 = 16 GT/s, Gen 5 = 32 GT/s. Each generation doubles the speed.
2. The width: the number of lanes (x1, x4, x8, x16).
3. Encoding overhead: Gen 3 to 5 lose about 1.5%; Gen 1 and 2 lost 20%.
4. The weakest part of the path: the link runs at the best speed and width that both ends and everything in between support. A Gen 5 card in a Gen 4 slot runs at Gen 4.
5. What the link actually trained to (LnkSta in lspci -vv), and how the slot is wired (an x16 slot may be wired x8).
6. Shared links: devices behind a PCIe switch share one uplink to the CPU.

Math for Gen 4 x8:

$16 \text{ GT/s} \times 8 \text{ lanes} = 128 \text{ Gb/s}$.

$128 / 8 \text{ bits per byte} = 16 \text{ GB/s}$.

Minus about 1.5% encoding = about 15.75 GB/s.

Answer: about 16 GB/s in each direction (about 32 GB/s in both directions together).

REMEMBER IT

$\text{GT/s} \times \text{lanes} / 8 = \text{GB/s each way}$. Gen 4 x8 = $16 \times 8 / 8 = 16 \text{ GB/s}$.

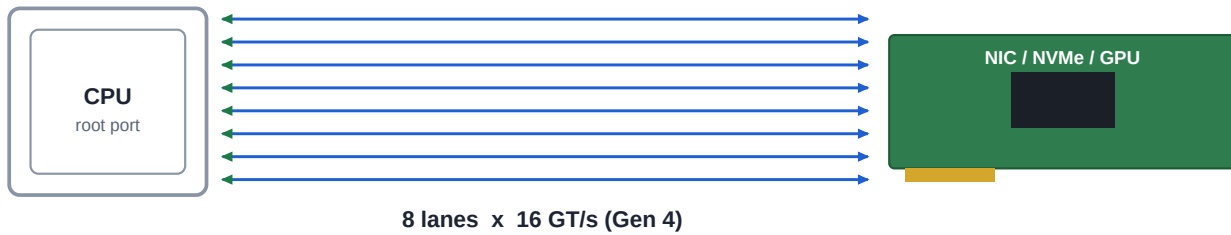
► **EXPLANATION** (understand it)

GT/s means "billions of transfers per second". Each transfer is one bit on one lane, so GT/s x lanes is roughly bits per second, and dividing by 8 turns bits into bytes.

Handy shortcut: Gen 4 is about 2 GB/s per lane each way, Gen 5 about 4 GB/s. So Gen 4 x8 = $8 \times 2 = 16 \text{ GB/s}$, and Gen 5 x8 equals Gen 4 x16 (about 32 GB/s).

SEE IT: TOPOLOGY / DIAGRAM

A GEN 4 X8 LINK: 8 LANES, EACH SENDING AND RECEIVING AT ONCE



$16 \text{ GT/s} \times 8 \text{ lanes} = 128 \text{ Gb/s}$ $128 / 8 \text{ bits} = 16 \text{ GB/s}$ minus ~1.5% encoding = ~15.75 GB/s

Answer: about 16 GB/s in EACH direction (about 32 GB/s both ways together)

SLOT SIZES (PHYSICAL)



Quick math: $\text{GT/s} \times \text{lanes} / 8 = \text{GB/s per direction}$. Generation sets speed per lane, width sets the number of lanes.

>_ TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Reading a Gen 4 x8 link

```
$ sudo lspci -vv -s 3b:00.0 | grep -E "LnkCap:|LnkSta:"
LnkCap: Port #0, Speed 16GT/s, Width x8, ASPM not supported
LnkSta: Speed 16GT/s, Width x8
```

→ 16 GT/s = Gen 4; x8 = 8 lanes. LnkSta matches LnkCap, so it got the full link: about 16 GB/s each way.

Topology tree versus diagram

Reread: Module 4: Topology diagram and topology tree

Your service guide includes a drawing of how every PCIe slot is wired, and Linux can print the PCIe tree it actually found. Explain what each one shows, which command prints the tree, and how comparing the two helps you find a fault.

ANSWER (simple words: write this)

The drawing is the topology diagram: the expected picture. It shows which CPU and root port each slot connects to, which PCIe switches, risers, retimers and cables sit in between, the expected link width, how slots map to bus numbers, and which devices share a path.

The tree is the actual picture: what Linux really found. The command is `lspci -tv`. It shows the root ports, the switches and the devices under them, with bus numbers like [17-1a].

Comparing them shows where the fault is, from what is missing:

- A port is there but nothing is under it: that one device did not link. Suspect the device, its power cable, its seating, the slot, or a retimer.
- A whole branch is missing: something upstream failed. Suspect the riser, cable, switch, retimer, the CPU root port, or a slot disabled in the BIOS.
- Everything under one switch is missing: suspect the switch or its uplink, not each device.

So instead of swapping parts one by one, I go straight to the part that the missing devices have in common.

REMEMBER IT

Diagram = expected. Tree (`lspci -tv`) = actual. The shape of what is missing points to the part.

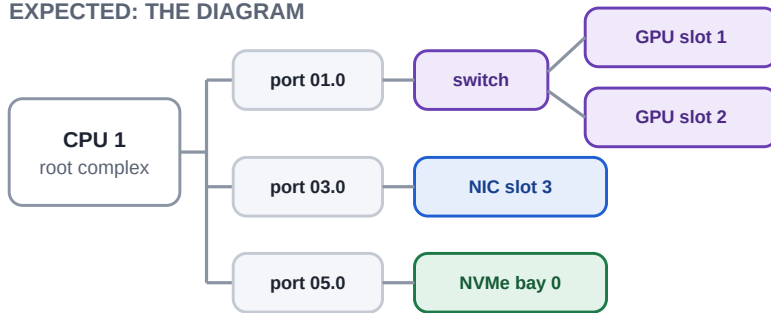
EXPLANATION (understand it)

It is like a seating chart for a theater. The chart (diagram) says who should be in each seat. Looking at the room (the tree) shows who is actually there. One empty seat means one person did not come; a whole empty row means the row's door is locked.

`lspci -vv` on the port also helps: "Physical Slot" tells you which slot it is, and PresDet (presence detect) tells you whether a card is even sensed.

SEE IT: TOPOLOGY / DIAGRAM

EXPECTED: THE DIAGRAM



The diagram shows:

- which CPU + root port per slot
- switches, risers, retimers, cables
- expected link width
- slot-to-bus-number mapping
- which devices share a path

ACTUAL: WHAT LINUX FOUND (LSPCI -TV)

```

-[0000:00]-+-01.0-[17-1a]----00.0-[18-1a]--+-00.0-[19]----00.0  NVIDIA GPU      (slot 1)
|
|          \-04.0-[1a]--              (empty!)
+-03.0-[3b]----00.0  Mellanox ConnectX-6 NIC
\ -05.0-[5e]----00.0  NVMe SSD
  
```

port, nothing under it

-> the device, its power, seating, slot

whole branch gone

-> riser, cable, switch, retimer, BIOS

all under one switch

-> the switch or its uplink

The diagram is what should be there; the tree is what Linux found. Here GPU slot 2's port (04.0) exists with nothing under it, so suspect that GPU, its power, seating or slot.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

The actual tree

```

$ lspci -tv
-[0000:00]-+-01.0-[17-1a]----00.0-[18-1a]--+-00.0-[19]----00.0  NVIDIA Corporation GH100 [H100 PCIe]
|
|          \-04.0-[1a]--
+-03.0-[3b]----00.0  Mellanox Technologies MT2892 Family [ConnectX-6 Dx]
\ -05.0-[5e]----00.0  Samsung Electronics Co Ltd NVMe SSD Controller
  
```

→ Port 04.0 has a bus number [1a] but nothing under it: GPU slot 2 did not link.

Ask the empty port which slot it is

```

$ sudo lspci -vv -s 18:04.0 | grep -E "Physical Slot|SltSta|LnkSta:"
Physical Slot: 2
LnkSta: Speed 2.5GT/s (downgraded), Width x0 (downgraded)
SltSta: Status: AttnBtn- PowerFlt- MRL- CmdCplt- PresDet+ Interlock-
  
```

→ Slot 2. PresDet+: a card is sensed in the slot, but the link is x0: it never trained. Suspect the GPU's power cable, seating, or a retimer.

What the BMC is and does

Reread: Module 8: The BMC

What is a BMC, and why does it keep working when the server's operating system has crashed? List at least six things a technician can do with it.

 **ANSWER** (simple words: write this)

The BMC (Baseboard Management Controller) is a small separate computer on the motherboard, with its own processor, firmware and network port. Vendor names: iDRAC (Dell), iLO (HPE), XCC (Lenovo), IPMI/BMC (Supermicro and others).

Why it keeps working when the OS has crashed: it does not depend on the OS at all. It runs on standby power, which is on whenever the server is plugged in, even when the server is off. It has its own network connection (the management network). This is called out-of-band management.

Things a technician can do with it (web page, ipmitool or Redfish):

1. Read the System Event Log (SEL): memory, PCIe, power, fan and temperature events with times (ipmitool sel elist).
2. Read sensors: temperatures, voltages, fan speeds, PSU status, power draw (ipmitool sensor).
3. See hardware inventory, health, serial numbers (ipmitool fru print) and firmware versions.
4. Use the virtual console (KVM) to see the screen.
5. Use Serial Over LAN for a text console (ipmitool sol activate).
6. Control power: on, off, reset, power cycle (ipmitool power cycle).
7. Send an NMI to force a crash dump.
8. Change the boot order and mount an ISO as virtual media.

REMEMBER IT

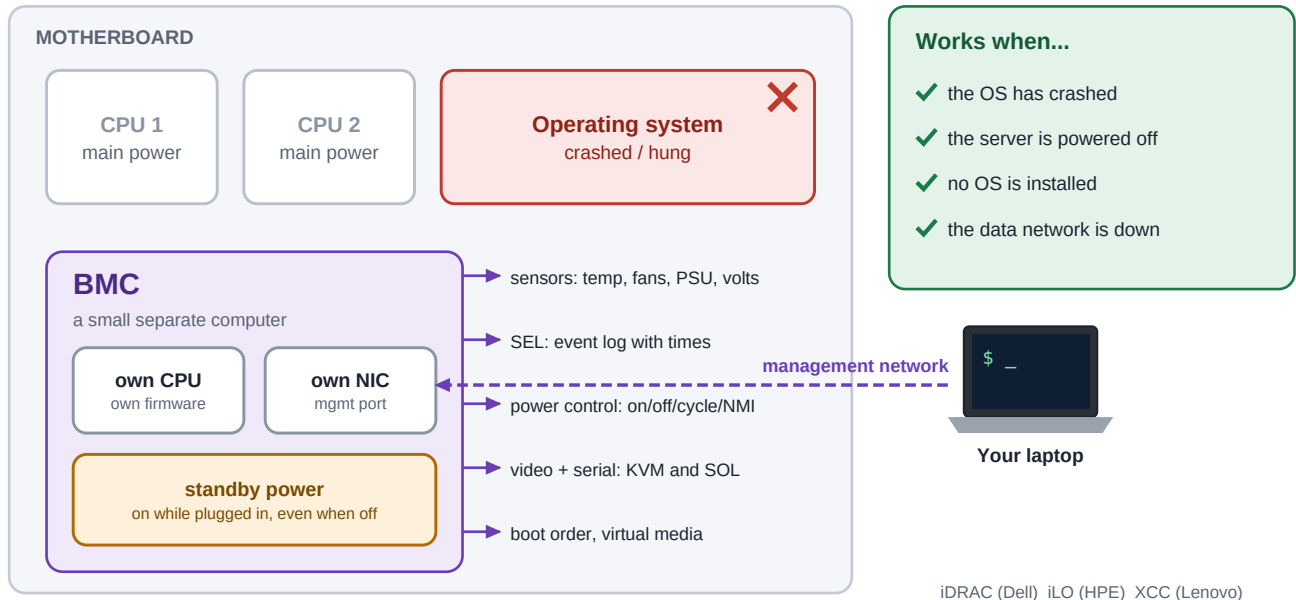
BMC = own CPU, own network port, standby power. Does: SEL, Sensors, Inventory, KVM, SOL, Power, NMI, Boot/ISO.

► **EXPLANATION** (understand it)

The BMC is like the night security guard of a building: separate from the office workers (the OS), on its own power line, with cameras (sensors), a logbook (SEL) and the keys to every door (power control). When the offices are dark, the guard is still there.

That is why the first step for a frozen or unreachable server is almost always "open the BMC".

SEE IT: TOPOLOGY / DIAGRAM



The BMC has its own processor, firmware, network port and standby power. It does not need the operating system, so it keeps working when the OS is gone.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Sensors and the event log

```
$ sudo ipmitool sensor | head -5
CPU1 Temp      | 58.000      | degrees C   | ok
Inlet Temp     | 24.000      | degrees C   | ok
FAN1           | 7800.000    | RPM         | ok
PSU1 Status    | 0x1         | discrete    | 0x0100| na
PSU2 Status    | 0x1         | discrete    | 0x0100| na

$ sudo ipmitool sel elist | tail -2
91 | 10/02/2026 | 03:12:44 | Memory DIMM_C1 | Correctable ECC | Asserted
92 | 10/02/2026 | 03:40:02 | Watchdog2      | Hard reset      | Asserted
```

→ Live readings, and a timeline of hardware events. These work even when the OS is down.

Inventory and power control

```
$ sudo ipmitool fru print 0 | grep -E "Product Name|Serial"
Board Serial      : WM21AS003127
Product Name      : SYS-421GE-TNRT
Product Serial    : S421A0991

$ sudo ipmitool power status
Chassis Power is on
```

→ Serial numbers for the repair record, and the power state.

LnkCap versus LnkSta

Reread: Module 4: Link training

Explain what LnkCap and LnkSta mean in lspci -vv output. What does it mean when they do not match, and what could cause it?

 **ANSWER** (simple words: write this)

LnkCap (link capability): the best speed and width the device can do, for example Speed 16GT/s (Gen 4), Width x16.

LnkSta (link status): what the link actually trained to when the system powered on.

At power on, every PCIe link negotiates ("trains") its speed and width. If LnkSta matches LnkCap, the device got its full link. If LnkSta is lower, the link is downtrained: the device works, but slower. For example LnkCap x16 and LnkSta x4 = a quarter of the bandwidth.

Possible causes:

- The slot or path cannot do more: a Gen 5 card in a Gen 4 slot, or an x16 slot that is only wired x8. This one is expected, not a fault (check the topology diagram).
- Poor seating or a dirty connector.
- A bad riser, cable or retimer.
- A BIOS setting: link speed forced lower, or the wrong bifurcation.
- Signal problems, which usually also show as AER errors in dmesg.

Fix one thing at a time (reseat, clean, swap the riser or cable) and check LnkSta again after each change.

REMEMBER IT

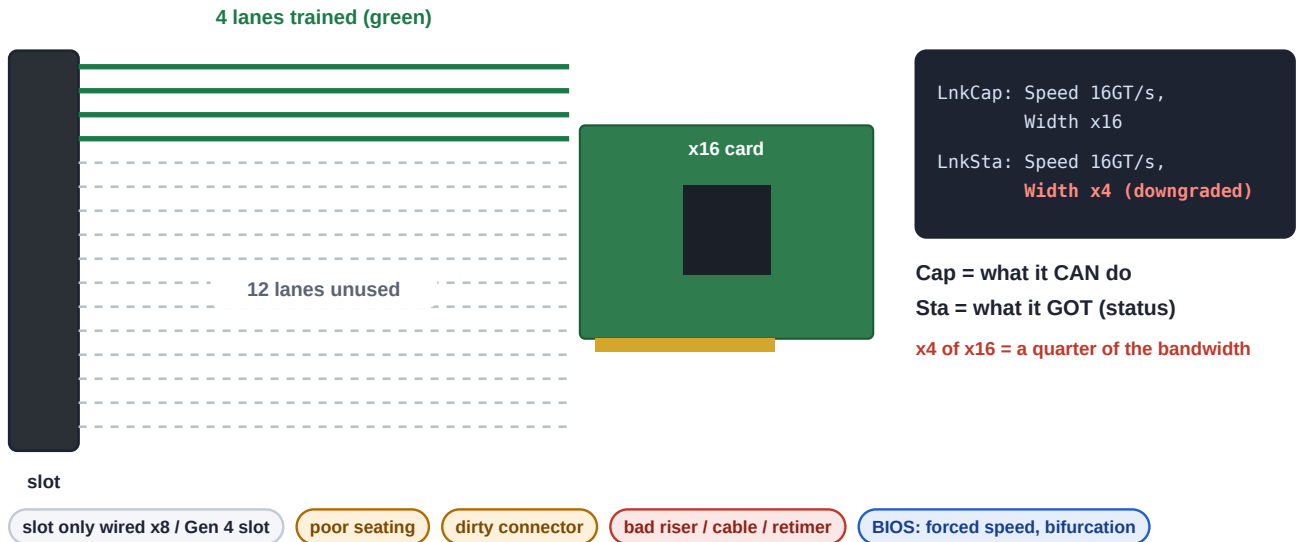
Cap = Can. Sta = Status (what it got). Sta lower than Cap = downtrained: works, but slower.

► **EXPLANATION** (understand it)

Link training is like two people agreeing how fast to talk on a noisy phone line. If the line is clean, they talk at full speed. If it is noisy, they slow down until they can understand each other. A downtrained link is a noisy line.

Modern lspci even prints "(downgraded)" next to the speed or width in LnkSta when it is lower than LnkCap.

SEE IT: TOPOLOGY / DIAGRAM



At power on each link trains its speed and width. LnkSta lower than LnkCap means the link is downtrained: it works, just slower.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

A downtrained x16 card

```
$ sudo lspci -vv -s 19:00.0 | grep -E "LnkCap:|LnkSta:"
LnkCap: Port #0, Speed 16GT/s, Width x16, ASPM not supported
LnkSta: Speed 16GT/s, Width x4 (downgraded)
```

→ Full speed (Gen 4) but only 4 of 16 lanes: a quarter of the bandwidth. Check the slot wiring first, then seating, riser, cable, retimer and BIOS.

Often seen together: corrected PCIe errors

```
$ dmesg | grep -c "AER: Corrected"
1832
```

→ A steady stream of corrected errors on the same path points to a signal problem, not a slot that is simply wired x4.

The four layers of a working device

Reread: Module 3: The four layers

Describe the four layers you check to decide whether a device is really working in Linux, and name the command or check you would use at each layer. Give an example where a device passes one layer and fails the next.

 **ANSWER** (simple words: write this)

1. Physical: is it installed, seated and powered, and does the BIOS or BMC see it? Check: look at it, its LEDs, and the BIOS or BMC inventory.
2. Bus: does the OS see it on the bus? Check: `lspci` for PCIe devices, `lsblk` or `nvme list` for drives.
3. Driver: is a driver attached? Check: `lspci -k` ("Kernel driver in use"), and `dmesg` for driver errors.
4. Function: is it configured and actually working? Check: `ip a` for a NIC, `nvidia-smi` for a GPU, or a functional test.

Example 1: a NIC shows in `lspci` (passes Bus), but `lspci -k` has no "Kernel driver in use" line (fails Driver). The hardware is fine; the driver is missing. Fix the driver; do not replace the card.

Example 2: a GPU shows in `lspci` (passes Bus) but not in `nvidia-smi` (fails Driver/Function). That is a driver or firmware problem, not hardware.

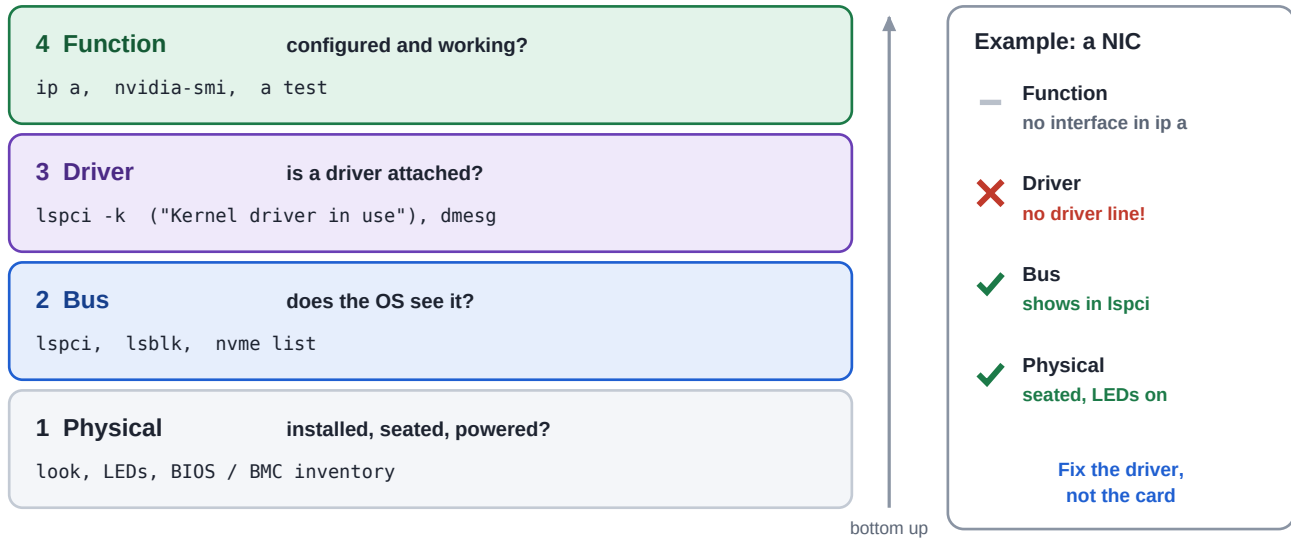
REMEMBER IT

P-B-D-F: Physical, Bus, Driver, Function ("Please Buy Donuts Friday"). Find the first layer that fails.

► **EXPLANATION** (understand it)

Each layer depends on the one below it. A card that is not seated cannot be on the bus; a device not on the bus cannot get a driver; without a driver it cannot work. So you check from the bottom up and stop at the first layer that fails: that is where the problem lives, and it tells you what kind of fix to make (hardware, driver or configuration).

SEE IT: TOPOLOGY / DIAGRAM



Physical, Bus, Driver, Function ("Please Buy Donuts Friday"). Find the first layer that fails: that is where the problem is.

>_ TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Passes Bus, fails Driver (a NIC)

```
$ lspci | grep -i ethernet
3b:00.0 Ethernet controller: Mellanox Technologies MT2892 Family [ConnectX-6 Dx]
$ lspci -k -s 3b:00.0
3b:00.0 Ethernet controller: Mellanox Technologies MT2892 Family [ConnectX-6 Dx]
Subsystem: Mellanox Technologies Device 0016
(no "Kernel driver in use" line)
$ modinfo mlx5_core
modinfo: ERROR: Module mlx5_core not found.
```

→ On the bus, but no driver installed: install the driver package. The card is fine.

Passes Bus, fails Function (a GPU)

```
$ lspci | grep -ic nvidia
8
$ nvidia-smi
NVIDIA-SMI has failed because it couldn't communicate with the NVIDIA driver.
Make sure that the latest NVIDIA driver is installed and running.
```

→ All 8 GPUs are on the bus, but the driver is not running: a driver problem, not 8 bad GPUs.

Symptom versus root cause

Reread: Module 10: Symptom versus root cause

A technician writes “Root cause: GPU not detected” on a repair ticket. Explain why that is a symptom and not a root cause. Then use the 5 Whys to trace an example of your own down to a process root cause.

 **ANSWER** (simple words: write this)

"GPU not detected" is what was observed, so it is the symptom. The root cause is why it happened. If you only fix the symptom (reseat the GPU and ship the unit), the same thing happens again on the next unit.

Example with the 5 Whys:

1. Why was the GPU not detected? Its power cable was not connected.
2. Why was it not connected? The cable was routed where it could not reach the GPU.
3. Why was it routed that way? The assembler followed the work instruction.
4. Why did the work instruction show that routing? It showed an old layout.
5. Why did it show an old layout? It was not updated after a design change to the chassis.

Root cause: the out-of-date work instruction, a process problem, not this one unit. Fix: update the work instruction, add a step so design changes always update the instructions, and check other units built with the old version.

REMEMBER IT

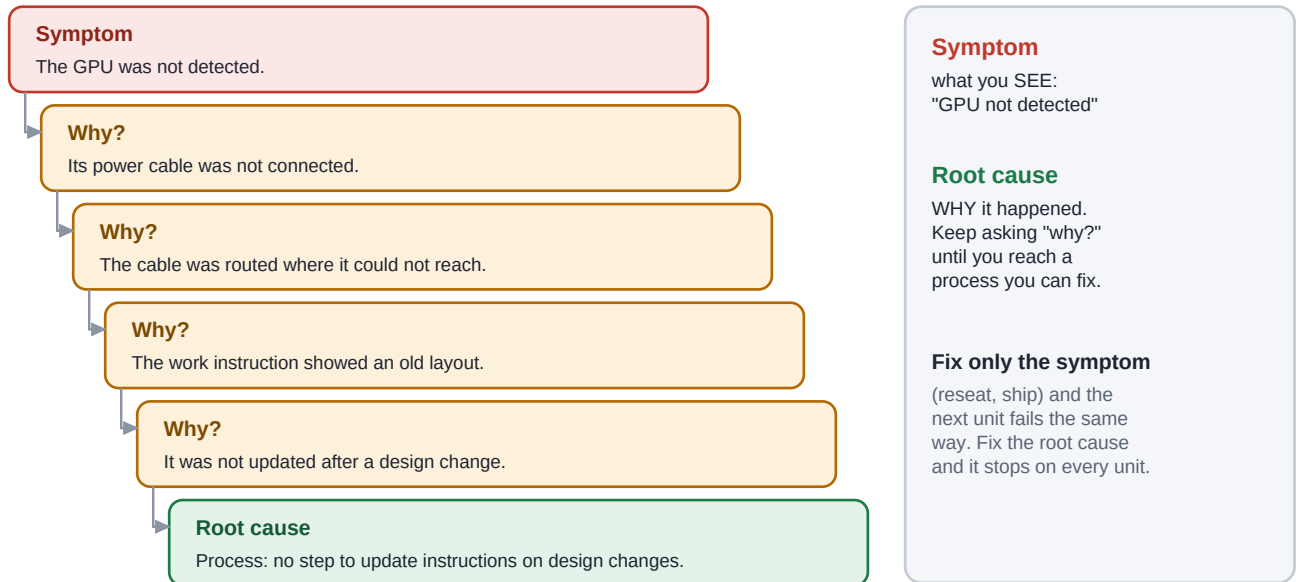
Symptom = what you see. Root cause = why. Ask "why?" until you reach a process.

► **EXPLANATION** (understand it)

A symptom tells you what is wrong; a root cause tells you how to stop it happening again. If a ticket's "root cause" could also be read as "the problem", it is still a symptom.

Stop asking "why?" when you reach something you can change in the process (an instruction, a check, a tool), not when you reach a person to blame.

SEE IT: TOPOLOGY / DIAGRAM



Each "why?" goes one level deeper. Stop when you reach a process, not a person or a single unit.

>_ TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

The symptom

```
$ nvidia-smi -L | wc -l
7
$ nvidia-smi -L | tail -2
GPU 5: NVIDIA H100 80GB HBM3 (UUID: GPU-6f1c...)
GPU 6: NVIDIA H100 80GB HBM3 (UUID: GPU-a207...)
```

→ 7 of 8 GPUs: that is what is wrong, not why.

Weak ticket vs strong ticket

```
WEAK   Root cause: GPU not detected. Action: resealed GPU.
STRONG Symptom:    GPU slot 7 not detected (7 of 8 in nvidia-smi).
        Cause:     aux power cable not connected; routing in WI rev C cannot reach.
        Root cause: WI not updated after chassis change ECO-1142.
        Action:    cable connected, all 8 GPUs pass; WI updated to rev D;
                   shift 2 units from the same build checked.
```

→ The strong ticket names the process problem and what was done about it.

Server lost network after a move

Reread: Module 7: A server that stops responding; Module 8

A Linux server stopped answering ping and SSH right after it was moved to a new rack position. What is your very first check, and what is the reason for it? Then walk through the rest of your process in order.

 ANSWER (simple words: write this)

First check: is the server actually up? Use the BMC (virtual console or SOL, and power status). Reason: you cannot SSH into a server that is off or crashed. After a move it may not have powered back on, may be stuck in POST, or may show an error. The BMC tells me if it is a server problem or a network problem before I start pulling cables.

Then, in order:

1. Physical link: cable plugged in at both ends, link lights on, transceiver seated, cable in the right switch port for the new position. `ethtool` shows "Link detected", `ip a` shows `LOWER_UP` or `NO-CARRIER`. Try a known-good cable or port.
2. NIC and driver: `lspci`, `lspci -k` and `dmesg`. A card or riser can come loose during a move.
3. IP address: `ip a`. Is there an address? 169.254 means DHCP failed, which happens if the new switch port is on a different VLAN.
4. Gateway: `ip route`, ping the gateway, then ping past it and use `traceroute`.
5. Names: test by IP and by name.
6. Service and firewall: `systemctl status sshd`, `ss -tulpn`, firewall rules.
7. Ask what changed with the move (new port, VLAN, cable), test from another machine, fix it, confirm ping and SSH work, and document every check.

REMEMBER IT

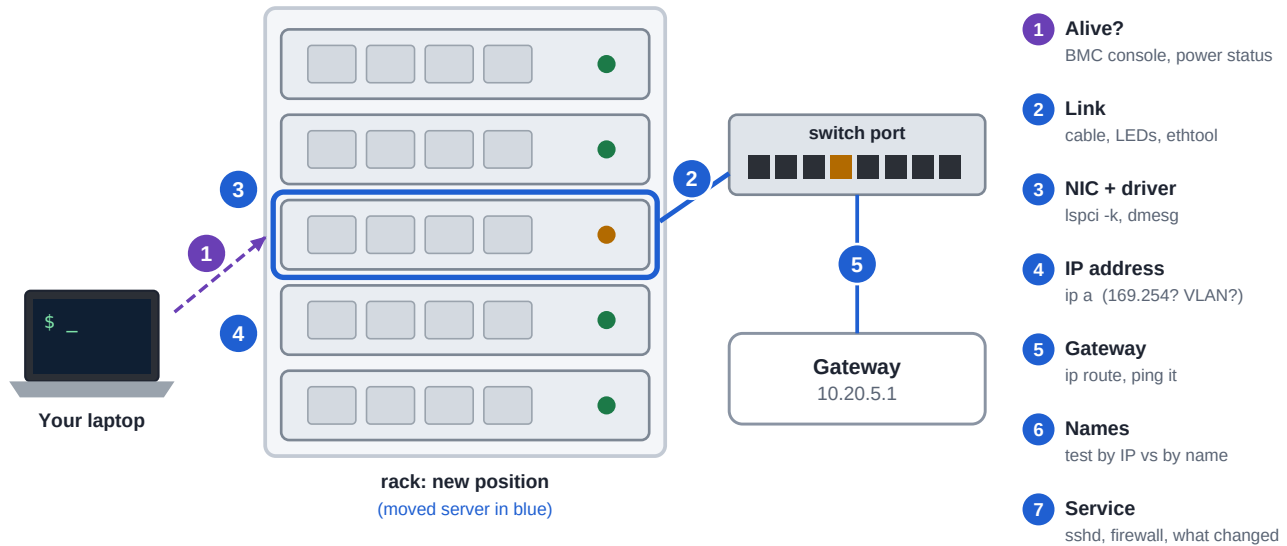
First: is it alive? Ask the BMC. Then bottom-up: Link, NIC, IP, Gateway, Name, Service.

► **EXPLANATION** (understand it)

A move changes physical things: power cables, network cables, the switch port and maybe the VLAN. So the most likely causes are at the bottom of the stack.

The BMC check comes first because it splits the problem in half in one minute: either the server is down (power, POST, boot), or it is up and the network path is the problem.

SEE IT: TOPOLOGY / DIAGRAM



First prove the server is alive through the BMC (you cannot SSH into a server that is off or crashed). Then work from the cable upward.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Step 0: the BMC says it is alive

```
$ ipmitool -I lanplus -H 10.20.9.14 -U admin -P ***** power status
Chassis Power is on
$ ipmitool -I lanplus -H 10.20.9.14 -U admin -P ***** sol activate
srv-r07-u12 login: _
```

→ Powered on and sitting at a login prompt: the OS is up, so this is a network problem.

Step 1 (on the console): no link

```
$ ip a show ens1f0 | head -1
2: ens1f0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 9000 qdisc mq state DOWN
$ ethtool ens1f0 | grep "Link detected"
Link detected: no
```

→ NO-CARRIER: no physical link. Check the cable at both ends and the switch port for the new position.

New card missing from lspci

Reread: Module 3: lspci; Module 4: Topology; Module 11: Playbook A

A technician installs a new 100G network card and says it is dead because lspci does not show it. What would you check, in order, before agreeing?

 **ANSWER** (simple words: write this)

1. Check we searched correctly: a 100G Mellanox card shows as "Ethernet controller" (or "Infiniband controller"). Run `lspci | grep -i -E "ethernet|mellanox|infiniband"` and compare the count with the build sheet.
2. BIOS and BMC inventory: if they do not see the card either, it is physical or firmware. If they see it but Linux does not, it is something else.
3. Physical, powered off with an ESD strap: card fully seated and latched, riser seated, riser cable connected, any power cable connected, no bent pins or damage, and the card in the slot the build calls for.
4. BIOS settings: slot enabled, bifurcation right for that slot, link speed not forced.
5. Topology: compare `lspci -tv` with the topology diagram. Port there but empty: card, seating, slot. Whole branch missing: riser, cable, switch, retimer. Are other devices on the same riser missing too?
6. Logs: `dmesg` and the BMC SEL for PCIe or hot-plug messages ("Card not present").
7. Swap: put the card in a known-good slot, or a known-good card in this slot. If the problem follows the card, the card is bad.

Only then agree it is dead: tag it, document it, start an RMA.

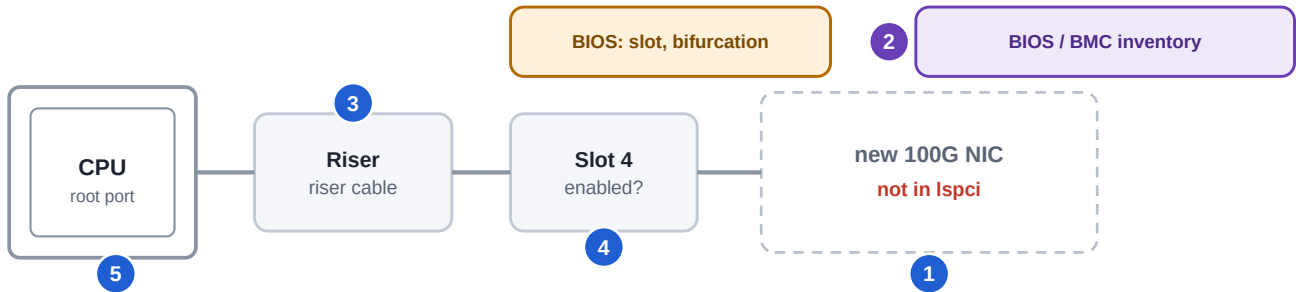
REMEMBER IT

Walk the whole path: Search, BIOS/BMC, Seat, Settings, Tree, Logs, Swap. Only then call it dead.

► **EXPLANATION** (understand it)

"Not in lspci" means the card never appeared on the bus. That can be the card, but it is just as often the slot, the riser, a cable or a BIOS setting. A new card being dead on arrival is the least likely cause, so it is the last thing you conclude, after the swap proves it.

SEE IT: TOPOLOGY / DIAGRAM



1 Search right: `lspci | grep -i -E "ethernet|mellanox"`

2 Does the BIOS or BMC inventory see it?

3 Physical (off, ESD): seated, latched, riser, power

4 BIOS: slot enabled, bifurcation, link speed

5 Topology: `lspci -tv` vs diagram

6 Logs: `dmesg`, BMC SEL for PCIe errors

7 Swap: known-good slot or card

Before calling a card dead, walk its whole path: how you searched, BIOS and BMC, seating, settings, the PCIe tree, the logs, and a swap.

>_ TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Searching the right way, then the tree

```
$ lspci | grep -i -E "ethernet|mellanox|infiniband"
c1:00.0 Ethernet controller: Intel Corporation Ethernet Controller X710 for 10GBASE-T
c1:00.1 Ethernet controller: Intel Corporation Ethernet Controller X710 for 10GBASE-T
(no Mellanox card)
$ lspci -tv | grep -A1 "30:02.0"
+-02.0-[31]--
```

→ Only the onboard 10G ports. The port that feeds slot 4 has a bus number but nothing under it.

The slot does not even sense a card

```
$ sudo lspci -vv -s 30:02.0 | grep -E "Physical Slot|SltSta"
Physical Slot: 4
SltSta: Status: AttnBtn- PowerFlt- MRL- CmdCplt- PresDet- Interlock-
$ dmesg | grep -i "slot(4)"
[ 6.211873] pciehp 0000:30:02.0: pciehp: Slot(4): Card not present
```

→ PresDet- and "Card not present": the slot does not sense the card at all. Reseat it (and its riser) before blaming the card.

Intermittent failure

Reread: Module 10: Intermittent problems

A test fails roughly once every 15 runs, and you cannot make it fail on demand. List the data you would gather, and explain how you would turn it into a failure you can reproduce.

 **ANSWER** (simple words: write this)

Data to gather:

- The exact error message and which test step fails, every time.
- The timestamps and how often (which run numbers failed).
- Which tests pass and which fail.
- Temperature and load at the time of each failure.
- Logs from each failure: dmesg, journalctl and the BMC SEL.
- Serial numbers, and firmware and driver versions.
- Whether other units or other slots show the same thing.

Turning it into something I can reproduce:

1. Look for a pattern: only under heavy load, only when hot, only late in a long run, only in one slot or one unit.
2. Run the test in a loop and log every run, for example for i in \$(seq 1 100); do ./test.sh >> results.txt; done, while watching dmesg -w and the sensors.
3. Push the suspected condition on purpose: more load, more heat, a longer run.
4. Look for corrected errors (PCIe AER, ECC) that warn about a weak part before it fails.
5. Check seating and cables, then change one variable at a time and rerun enough cycles to trust the result. At 1 failure in 15, twenty clean runs proves nothing; I want 100 or more.
6. Compare with a golden unit, and document every step and result.

REMEMBER IT

Log every run, find the pattern, then cause it on purpose. Loop + watch + one change at a time.

► EXPLANATION (understand it)

An intermittent failure is not random; you just have not found its trigger yet. Logging every run lets you line up the failures with something else (temperature, load, time, a slot).

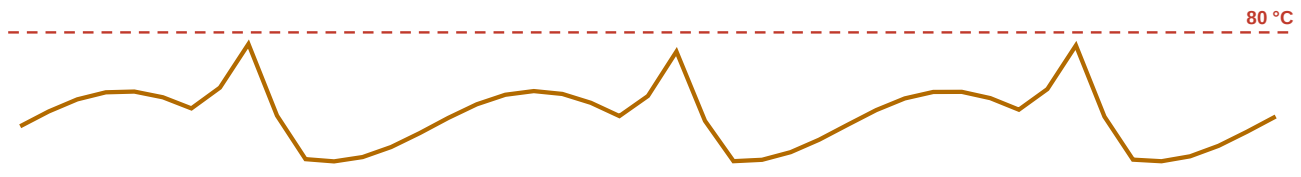
Once you can make it fail on purpose, you can test a fix properly: if it used to fail 1 in 15 and now passes 150 in a row, the fix worked.

SEE IT: TOPOLOGY / DIAGRAM

RUNS 1 TO 45 (LOG EVERY RUN)



GPU TEMPERATURE DURING EACH RUN



Pattern: it fails only when the GPU passes 80 °C. Now you can reproduce it on purpose (more load, more heat).

```
for i in $(seq 1 100); do ./test.sh >> results.txt; done
```

Watch dmesg -w and the sensors while it loops. At 1 failure in 15 runs, you need 100+ clean runs to trust a fix.

Turn "random" into a pattern: log every run, line it up with temperature, load and time, then make the condition happen on purpose.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Which runs failed

```
$ grep -c FAIL results.txt
3
$ grep -n FAIL results.txt
9:run 9  FAIL  2026-10-02 10:41:07 dma_test: timeout on GPU 3
24:run 24  FAIL  2026-10-02 12:58:44 dma_test: timeout on GPU 3
38:run 38  FAIL  2026-10-02 15:02:19 dma_test: timeout on GPU 3
```

→ Always the same GPU and the same step: a pattern already.

What happened at that exact time

```
$ journalctl -k --since "10:40:30" --until "10:41:30" | tail -2
Oct 02 10:41:05 srv-r07-u12 kernel: pcieport 0000:17:01.0: AER: Corrected error message received from 0000:3d:00.0
Oct 02 10:41:07 srv-r07-u12 kernel: NVRM: Xid (PCI:0000:3d:00): 79, pid=0, GPU has fallen off the bus.
$ grep "10:41" sensors_log.txt | grep GPU3
10:41:00 GPU3 Temp 83 C
```

→ Corrected PCIe errors just before each failure, and GPU 3 is over 80 C: suspect heat plus a weak link. Now reproduce it with heat on purpose.

Powers on but no OS

Reread: Module 9: Powers on but will not boot

A server's fans spin and its health LED is green, but you never get a login prompt. Describe your troubleshooting stage by stage, and where you would look at each stage.

 **ANSWER** (simple words: write this)

Fans and a green LED only prove the server has power. Booting is a chain, so I find how far it gets. I open the console through the BMC (virtual console or SOL), watch a whole boot, and note exactly where it stops. I also check the BMC event log for CPU, memory, PCIe or power errors.

1. POST: stuck on a POST code or beeping? Memory training on a large server takes minutes, so wait first. If truly stuck, look up the code and suspect memory, a CPU or a PCIe card; reseal. If needed, strip to a minimum configuration (one CPU, one DIMM, no cards) and add parts back one at a time.
2. UEFI/BIOS: "No bootable device" or a PXE loop? Check the boot drive appears in the BIOS, the boot order, UEFI versus Legacy, Secure Boot, the cables and the RAID controller.
3. Bootloader (GRUB): a grub rescue> prompt or "file not found" points to the boot partition or the drive.
4. Kernel: a kernel panic or "Unable to mount root fs" points to a storage driver, the root filesystem or the drive. Read the exact message.
5. System startup: emergency mode usually means a bad /etc/fstab entry or a failing disk. Log in on the console and read journalctl -xb.
6. If the console reaches a login prompt but I still cannot log in over the network, it booted and the problem is the network.

Always ask what changed, check firmware, and document where it stopped and what fixed it.

REMEMBER IT

Booting is a chain: Power, POST, BIOS, GRUB, Kernel, Startup. Watch the console and find where it stops.

► **EXPLANATION** (understand it)

Think of the boot chain as a relay race: each runner hands the baton to the next. The console shows you exactly which runner dropped it. Without the console you are guessing; with it, the exact message usually tells you which module of the course to use next.

SEE IT: TOPOLOGY / DIAGRAM

STAGE	FAILURE LOOKS LIKE	WHERE TO LOOK
1 Power + BMC	no lights, shuts off, amber LED	PSUs, cords, BMC SEL
2 POST	stuck POST code, beeps, memory errors	POST code, SEL; wait for memory training
3 UEFI / BIOS	"No bootable device", PXE loop	BIOS: drive seen? boot order, UEFI/Legacy
4 Bootloader (GRUB)	grub rescue>, file not found	boot partition, the drive
5 Kernel	kernel panic, cannot mount root fs	read the exact message; storage driver
6 System startup	emergency mode	/etc/fstab, failing disk: journalctl -xb

Watch the console (BMC virtual console or SOL) and note exactly where it stops. Fans and LEDs only prove power.

Booting is a chain. The stage where it stops tells you which link broke and where to look next.

>_ TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Watching the boot over SOL: it stops in GRUB

```
$ ipmitool -I lanplus -H 10.20.9.14 -U admin -P ***** sol activate
[SOL Session operational. Use ~? for help]
error: no such partition.
Entering rescue mode...
grub rescue> _
```

→ POST and the BIOS worked (it found a disk and started GRUB). GRUB cannot find its partition: look at the boot drive and partition, and ask what changed.

The BMC log for the same boot

```
$ ipmitool -I lanplus -H 10.20.9.14 -U admin -P ***** sel elist | tail -2
a3 | 10/02/2026 | 08:05:44 | System Firmware Progress | Starting operating system boot process | Asserted
a4 | 10/02/2026 | 08:05:45 | Drive Slot 0 | Drive Fault | Asserted
```

→ A drive fault in slot 0 (the boot drive) right at boot: a strong lead.

NVMe drops out under load

Reread: Module 6: A drive that drops out

During a long stress test, an NVMe drive vanishes from lsblk. After a reboot it is back and looks healthy. What evidence would you look for, and what could be causing it?

 **ANSWER** (simple words: write this)

A reboot resets the link, so the drive comes back, but the cause is still there.

Evidence:

1. Kernel messages from when it dropped. If not rebooted yet: `save dmesg > nvme_drop.txt` first. After a reboot: `journalctl -b -1 -k | grep -i nvme`. Look for "I/O timeout, aborting", "controller is down; will reset", "Removing after probe failure", and PCIe AER errors.
2. The BMC event log for drive, PCIe or temperature events at the same time.
3. Drive health: `nvme smart-log` (temperature, critical warnings, media errors, unsafe shutdowns).
4. The link: `sudo lspci -vv` for LnkSta versus LnkCap, and AER errors on the port above the drive.
5. Firmware versions for the drive, the backplane and the BIOS.

Possible causes: poor seating or a worn backplane connector, a bad or misrouted cable, overheating under load, a power-saving feature that does not wake up, a firmware bug, a PCIe signal problem, or a failing drive.

Then isolate: move the drive to another bay. Follows the drive = the drive. Stays with the bay = the bay, backplane or cable. Reproduce under load (fio) while watching `dmesg -w` and the temperature, and document everything.

REMEMBER IT

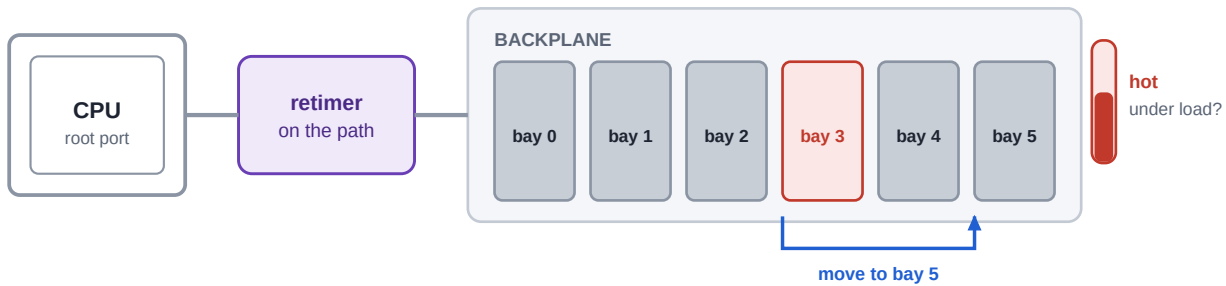
A reboot hides it, it does not fix it. Evidence: kernel log, SEL, smart-log, LnkSta, firmware. Then swap bays.

► **EXPLANATION** (understand it)

The test only fails "under load" and "after a long time": both point to heat, power or a weak signal that gives up when stressed. That is why temperature and AER errors are as important as the drive's own health.

The previous-boot journal is the key: `dmesg` after the reboot shows a healthy drive, but `journalctl -b -1` shows the moment it fell off.

SEE IT: TOPOLOGY / DIAGRAM



EVIDENCE TO COLLECT

Kernel log

```
journalctl -b -1 -k | grep -i nvme
```

BMC SEL

```
ipmitool sel elist  
(drive, PCIe, temp)
```

Health

```
nvme smart-log  
/dev/nvme1
```

Link

```
lspci -vv: LnkSta,  
AER errors
```

Firmware

```
drive, backplane,  
BIOS versions
```

Follows the drive to bay 5: the drive. Stays with bay 3: the bay, backplane connector or cable.

A reboot resets the link, so the drive comes back. The cause is still there.

Save the kernel messages before rebooting (or read the previous boot after), then check heat, health, the link and firmware, and swap bays to isolate.

>_ TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

The previous boot shows the drop

```
$ journalctl -b -1 -k | grep -i nvme | tail -3  
Oct 02 03:14:51 srv-r07-u12 kernel: nvme nvme1: I/O 412 QID 9 timeout, aborting  
Oct 02 03:15:22 srv-r07-u12 kernel: nvme nvme1: controller is down; will reset: CSTS=0xffffffff,  
PCI_STATUS=0x10  
Oct 02 03:15:23 srv-r07-u12 kernel: nvme 0000:5e:00.0: Removing after probe failure status: -19
```

→ Timeout, then the controller stopped answering, then Linux removed it.

Health and temperature

```
$ sudo nvme smart-log /dev/nvme1 | grep -i -E "critical|^temperature|media|unsafe"  
critical_warning          : 0x2  
temperature               : 82 C (355 Kelvin)  
media_errors              : 0  
unsafe_shutdowns          : 3
```

→ critical_warning 0x2 = temperature warning; 82 C is hot. Media errors are 0. Suspect airflow in that bay before the drive.

GPU missing, empty port in lspci -tv

Reread: Module 11: Playbook B; Module 4: Topology

An 8-GPU server shows only seven GPUs. In lspci -tv, the PCIe port that should lead to GPU 6 is present but has nothing under it. List what could cause this, and explain how you would narrow it down.

 ANSWER (simple words: write this)

An empty port means the port exists but the device under it never linked. The path up to that port is working (otherwise the whole branch would be missing), so the problem is at the GPU end.

Possible causes:

- The GPU is not fully seated.
- Its auxiliary power cable is missing, loose or in the wrong connector.
- A bad slot or connector.
- A failed retimer on that path (on the riser or GPU baseboard).
- The slot disabled in the BIOS.
- The GPU itself is dead.

Narrowing it down:

1. Use the topology diagram to confirm which slot is GPU 6, and check the BMC inventory and SEL for power or PCIe events on it.
2. Check dmesg for PCIe errors on that port, and the BIOS slot settings.
3. Powered off with ESD protection: reseal GPU 6 and check its power cable is in the right connector. Check PSU status.
4. Swap test: move GPU 6 to a working slot, and put a known-good GPU in slot 6. Follows the GPU = bad GPU. Stays with slot 6 = slot, power cable or retimer.
5. Verify all eight GPUs show in lspci and nvidia-smi, run a full GPU test, and document.

REMEMBER IT

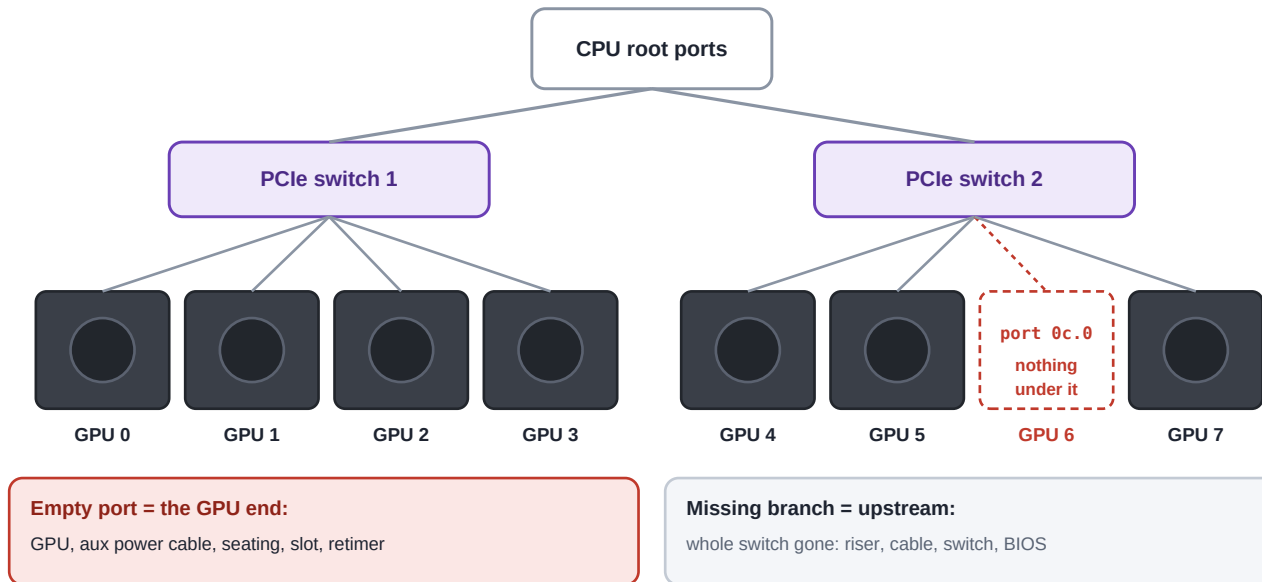
Empty port = the GPU end (GPU, power cable, seating, slot, retimer). Missing branch = upstream.

► **EXPLANATION** (understand it)

An empty port is like a mailbox with your name on it but no mail: the address exists, but nothing arrived. So the street (switch, riser) is fine; the problem is at your house (the GPU, its power, its seating).

lspci -vv on the empty port helps: PresDet+ means the slot senses a card, so think power cable and seating; PresDet- means no card is sensed at all.

SEE IT: TOPOLOGY / DIAGRAM



Seven of eight GPUs. The port for GPU 6 exists, so the path is fine up to the port: look at the GPU, its power cable, its seating and the slot.

>_ TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Counting at each layer

```
$ nvidia-smi -L | wc -l
7
$ lspci | grep -ic nvidia
7
```

→ Seven at the driver layer and seven on the bus: the eighth never reached the bus. A hardware path problem.

The empty port

```
$ lspci -tv | grep -B1 -A1 "0c.0"
|          +-08.0-[cb]----00.0  NVIDIA Corporation GH100 [H100 SXM5 80GB]
|          +-0c.0-[cc]--
|          \-10.0-[cd]----00.0  NVIDIA Corporation GH100 [H100 SXM5 80GB]
$ sudo lspci -vv -s c9:0c.0 | grep -E "SltSta|LnkSta:"
LnkSta: Speed 2.5GT/s (downgraded), Width x0 (downgraded)
SltSta: Status: AttnBtn- PowerFlt- MRL- CmdCplt- PresDet+ Interlock-
```

→ Port 0c.0 is empty. PresDet+ (a card is sensed) but no link: check GPU 6's power cable and seating first.

Assembly cable error on three units

Reread: Module 11: Playbook D

Three servers from the same shift fail test because a cable was plugged into the wrong connector during assembly. After fixing them, what do you do, and how do you keep it from happening again?

 ANSWER (simple words: write this)

1. Verify: each of the three servers fully passes test after the fix, and the wrong connection did not damage anything.
2. Document: the unit serials, which cable, the wrong port and the correct port, the test results, and photos if allowed.
3. Notify: my lead, quality and the assembly team.
4. Contain: three units from one shift means there may be more. Check other units from the same line, shift or batch, including units that already passed and units waiting to ship.
5. Prevent it:
 - Update the work instruction with clear pictures of the right connection.
 - Add labels or color coding on the cable and the connector.
 - Use mistake-proofing: keyed connectors, or cable lengths that only reach the right port.
 - Add an inspection step that checks this cable before test.
6. Track it through the site's corrective action process (CAPA) so it gets followed up.

The focus is on fixing the process, not blaming the person who plugged it in.

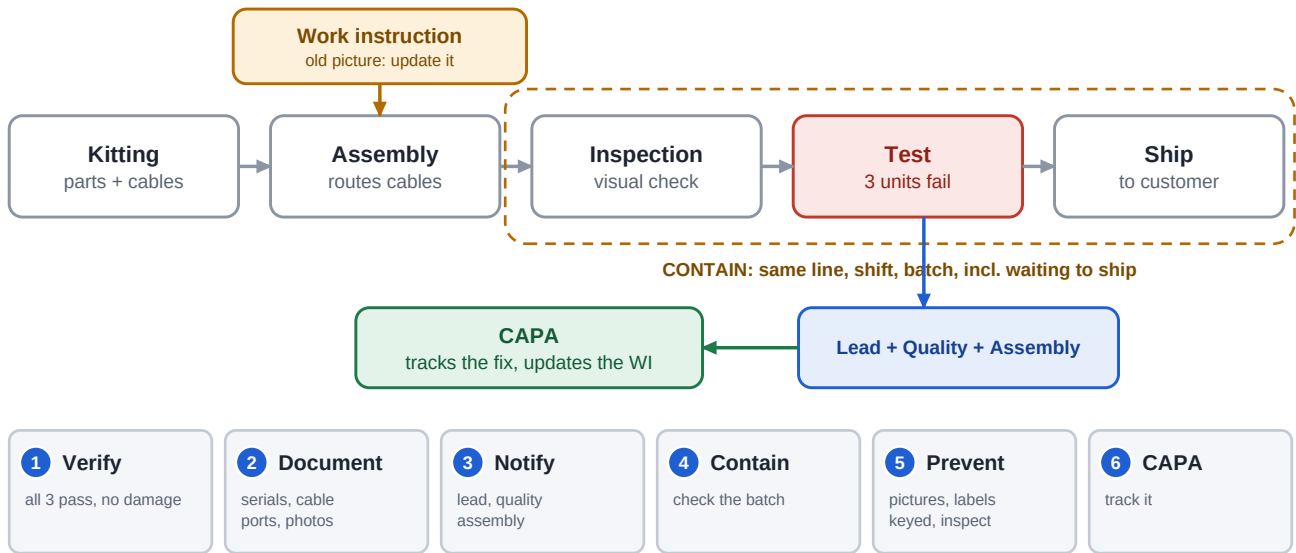
REMEMBER IT

V-D-N-C-P + CAPA: Verify, Document, Notify, Contain, Prevent. Fix the process, not the person.

► **EXPLANATION** (understand it)

Fixing the three units fixes today. Containment protects the customer from units you have not found yet, and prevention protects tomorrow. If the same mistake happened three times in one shift, the process made it easy to do; change the process so it becomes hard or impossible.

SEE IT: TOPOLOGY / DIAGRAM



Fix the process, not the person.

Fixing three units is not the end. Contain the rest of the batch, then change the process so the wrong connection cannot happen again.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Test logs: same failure, same shift

```

$ grep -H "RESULT" /testlogs/2026-10-01/shift2/*.log
J7K2Q38.log: RESULT PASS
J7K2Q39.log: RESULT FAIL nvme: 4 of 8 drives missing (bays 4-7)
J7K2Q40.log: RESULT PASS
J7K2Q41.log: RESULT FAIL nvme: 4 of 8 drives missing (bays 4-7)
J7K2Q42.log: RESULT FAIL nvme: 4 of 8 drives missing (bays 4-7)

```

→ Same symptom on three units from one shift: bays 4-7 = one backplane cable, plugged into the wrong connector.

Containment: units that passed or are waiting to ship

```

$ grep -l "shift2" /travelers/2026-10-01/*.txt | xargs grep -H "STATUS"
J7K2Q38.txt: STATUS WAITING TO SHIP
J7K2Q40.txt: STATUS WAITING TO SHIP
J7K2Q44.txt: STATUS IN TEST

```

→ Inspect these units' cable too, even though they passed or have not been tested yet.

grep across a folder

Reread: Module 2: Searching with grep

Write one command that searches every log file under `/var/log`, including subfolders, for the word `nvme`, ignoring upper and lower case and showing line numbers. Explain what each option does, and how you would count the matches instead.

 **ANSWER** (simple words: write this)

```
sudo grep -rin "nvme" /var/log/
```

- `-r`: recursive: search the folder and every subfolder.
- `-i`: ignore case: matches `nvme`, `NVMe` and `NVME`.
- `-n`: show the line number of each match (with `-r` it also shows the file name).
- `"nvme"` is the word to find, `/var/log/` is where to look. `sudo` because some logs are readable only by root.

To count instead:

- `sudo grep -ric "nvme" /var/log/` : swap `-n` for `-c` to get the number of matching lines in each file.
- `sudo grep -ri "nvme" /var/log/ | wc -l` : one total number for everything.

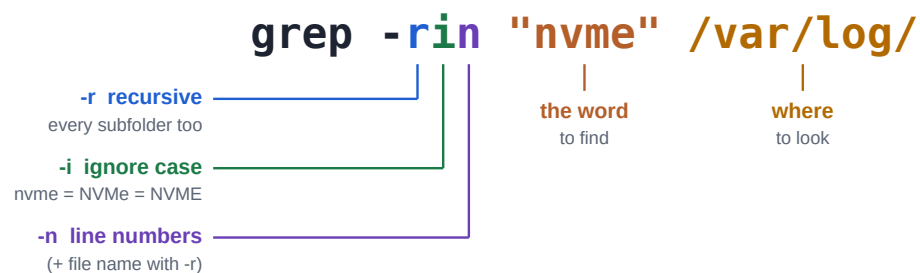
REMEMBER IT

`grep -rin` = Recursive, Ignore case, line Numbers. Count: `-c` or `| wc -l`.

EXPLANATION (understand it)

Each output line looks like `file:line-number:the line`, so you can open the file at exactly that line. The journal itself is a binary file, so `grep` finds text logs (messages, `dmesg`, `syslog`); for the journal use `journalctl | grep -i nvme`.

SEE IT: TOPOLOGY / DIAGRAM

**GREP -R WALKS THE WHOLE TREE**

```
/var/log/
  messages      match
  boot.log
  dmesg          match
  anaconda/
    syslog      match
  audit/
    audit.log
```

Count instead of listing

```
grep -ric "nvme" /var/log/
```

count per file (`-c` replaces `-n`)

```
grep -ri "nvme" /var/log/ | wc -l
```

one total for everything

Read the options as "R-I-N": recursive, ignore case, numbers. Swap `-n` for `-c` to count. Add `sudo`: some logs are readable only by root.

>_ TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

List the matches

```
$ sudo grep -rin "nvme" /var/log/ | head -3
/var/log/messages:48211:Oct  2 03:14:51 srv-r07-u12 kernel: nvme nvme1: I/O 412 QID 9 timeout, aborting
/var/log/messages:48233:Oct  2 03:15:22 srv-r07-u12 kernel: nvme nvme1: controller is down; will reset
/var/log/anaconda/syslog:902:[    3.114] NVMe drive /dev/nvme0n1 detected
```

→ file : line number : text. The last match is "NVMe" in capitals, found because of -i.

Count them

```
$ sudo grep -ric "nvme" /var/log/ | grep -v ":0$"
/var/log/messages:57
/var/log/anaconda/syslog:12
/var/log/dmesg:9
$ sudo grep -ri "nvme" /var/log/ | wc -l
78
```

→ Per file with -c (the extra grep hides files with 0), or one total with wc -l.

dmesg versus journalctl -b -1

Reread: Module 2: Kernel and system logs

A server crashed overnight and rebooted itself. Which one shows you what happened before the crash, dmesg or journalctl -b -1? Explain what each one shows, and what you should save before you reboot a failing server.

 **ANSWER** (simple words: write this)

journalctl -b -1 shows what happened before the crash.

dmesg shows kernel messages from the current boot only. It reads a buffer in memory that is cleared at every reboot, so after the crash and reboot, the messages from before the crash are gone from dmesg.

journalctl reads the systemd journal. -b -1 means the previous boot, so it shows the messages leading up to the crash (add -k for kernel messages only). This only works if the journal is kept on disk (persistent); if it says there is no previous boot, that evidence is lost.

If the journal just stops with no shutdown messages, that points to a sudden power loss or hardware reset, so I also check the BMC event log for the same time.

Before rebooting a failing server, save:

- The kernel messages: `dmesg -T > before_reboot.txt`
- The journal: `journalctl -b > journal_now.txt`
- The BMC event log and sensors (`ipmitool sel elist`, `ipmitool sensor`).
- A screenshot of the console if it shows an error.

Then copy the files off the server before the reboot.

REMEMBER IT

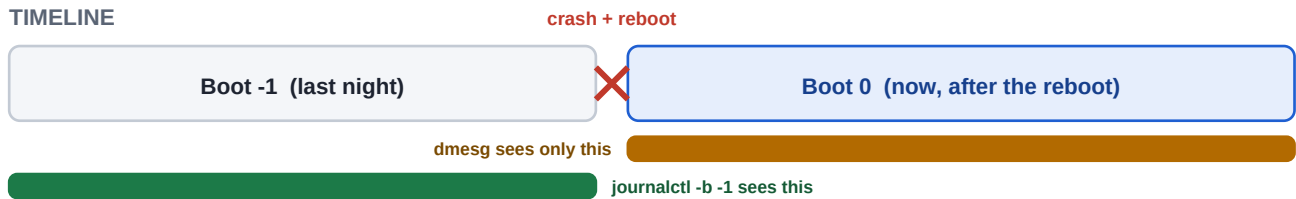
dmesg = whiteboard (wiped every reboot). journalctl -b -1 = notebook (last boot is still there).

► **EXPLANATION** (understand it)

After a crash, "dmesg" is the most common wrong answer: it looks at the new boot, which is healthy. journalctl --list-boots shows every boot the journal remembers; -1 is the one before this one, -2 the one before that.

SEE IT: TOPOLOGY / DIAGRAM

TIMELINE



dmesg = a whiteboard (RAM buffer)

Shows kernel messages from THIS boot only. Wiped at every reboot, so the messages from before the crash are already gone.

journalctl -b -1 = a notebook (journal on disk)

Reads the PREVIOUS boot from the journal on disk: the last messages before the crash (if the journal is persistent). Add -k for kernel messages only.

Before rebooting a failing server: `dmesg -T > before_reboot.txt` + BMC SEL + sensors + console screenshot

After a crash and reboot, dmesg only shows the new boot. `journalctl -b -1` shows what happened before the crash.

>_ TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

dmesg starts fresh at the new boot

```
$ dmesg -T | head -1
[Fri Oct 2 03:42:10 2026] Linux version 5.14.0-427.13.1.el9_4.x86_64 ...
$ journalctl --list-boots | tail -2
-1 6a1f... Thu 2026-10-01 08:02:11 EDT  Fri 2026-10-02 03:40:01 EDT
 0 c93e... Fri 2026-10-02 03:42:10 EDT  Fri 2026-10-02 09:14:02 EDT
```

→ dmesg begins at 03:42 (the new boot). The previous boot ended at 03:40.

The last messages before the crash

```
$ journalctl -b -1 -n 3 --no-pager
Oct 02 03:39:58 srv-r07-u12 kernel: mce: [Hardware Error]: CPU 12: Machine Check: 0 Bank 7
Oct 02 03:39:58 srv-r07-u12 kernel: mce: [Hardware Error]: TSC 0 ADDR 5f2a3240 MISC 8c Uncorrected
Oct 02 03:40:01 srv-r07-u12 kernel: Kernel panic - not syncing: Fatal machine check
```

→ An uncorrected machine check (often memory) caused the panic. dmesg could not show you this.

IP works but name fails, and the reverse

Reread: Module 7: DNS; A server that stops responding

Compare these two cases: (a) ping 10.4.2.15 works but ping db01 fails with “Name or service not known”; (b) ping db01 resolves to 10.4.2.15 but gets no replies. What does each result point to, and what would you check in each case?

 **ANSWER** (simple words: write this)

(a) The IP works, so the network is fine. The problem is name resolution (DNS). Check:

- /etc/resolv.conf: is the right DNS server listed?
- Can I reach that DNS server (ping it)?
- nslookup db01 or dig db01 to ask DNS directly.
- /etc/hosts for a missing or wrong entry.
- Whether I need the full name (FQDN, for example db01.lab.local) or the right search domain.

(b) The name resolved, so DNS is working. The problem is reaching the address, or db01 itself. Check:

- Is 10.4.2.15 still the right address for db01? An old DNS record or hosts entry could point to the wrong place.
- My own side: ip a, ip route, and ping the gateway.
- traceroute 10.4.2.15 to see where the traffic stops.
- Is db01 actually up? Check its console or BMC.
- Is a firewall on db01 or in between blocking ping?

REMEMBER IT

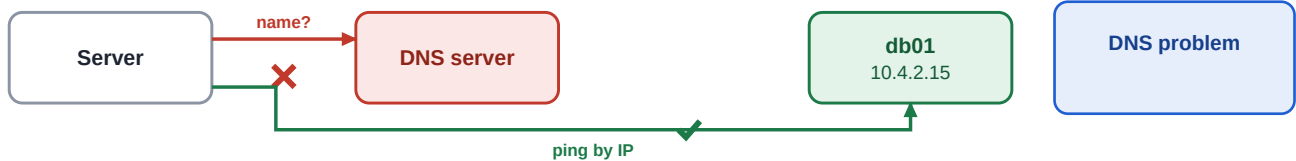
IP works, name fails = DNS. Name works, IP fails = network or target.

► **EXPLANATION** (understand it)

Separate the two jobs: "turn the name into a number" (DNS) and "deliver packets to that number" (the network). Each case tells you which job failed, so you only troubleshoot half the problem.

SEE IT: TOPOLOGY / DIAGRAM

(a) ping 10.4.2.15 works, ping db01: "Name or service not known"



Check: resolv.conf, DNS reachable, nslookup / dig, /etc/hosts, FQDN

(b) db01 resolves to 10.4.2.15, but no replies



Check: right IP? ip route, ping gateway, traceroute, is db01 up, firewall

IP works but the name fails: the network is fine and DNS is the problem. The name works but the IP does not answer: DNS is fine and the path or the target is the problem.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Case (a): the network works, DNS does not

```
$ ping -c 1 10.4.2.15
64 bytes from 10.4.2.15: icmp_seq=1 ttl=63 time=0.52 ms
$ ping -c 1 db01
ping: db01: Name or service not known
$ nslookup db01
;; connection timed out; no servers could be reached
```

→ The DNS server itself cannot be reached: check resolv.conf and whether 10.20.5.5 is up.

Case (b): DNS works, the path does not

```
$ ping -c 3 db01
PING db01.lab.local (10.4.2.15) 56(84) bytes of data.
--- db01.lab.local ping statistics ---
3 packets transmitted, 0 received, 100% packet loss
$ traceroute -n 10.4.2.15
 1  10.20.4.1  0.41 ms  0.38 ms  0.36 ms
 2  * * *
```

→ The name resolved, but packets stop after the gateway: a routing or firewall problem, or db01 is down.

Saving evidence before a reboot

Reread: Module 2: Kernel and system logs; Module 8

A failing server must be power cycled in ten minutes. Write the commands you would run first to save the evidence to files, and explain what each file would show an engineer later.

 **ANSWER** (simple words: write this)

Commands (name each file with the unit serial):

1. `dmesg -T > unit123_dmesg.txt`

Kernel messages from this boot, with readable times: hardware errors, driver problems, PCIe and memory errors. Lost at the reboot.

2. `journalctl -b > unit123_journal.txt`

Everything logged this boot (kernel and services), to see what happened and in what order.

3. `journalctl -b -1 > unit123_prevboot.txt`

The boot before, if the problem started then.

4. `sudo ipmitool sel elist > unit123_sel.txt`

The BMC event log: memory, PCIe, power, fan and temperature events with times.

5. `sudo ipmitool sensor > unit123_sensors.txt`

Temperatures, fan speeds, PSU status and voltages at the moment of failure.

6. `sudo lspci -vv > unit123_lspci.txt`

Every PCIe device and its link status: which devices were present and how they trained.

7. `sudo dmidecode > unit123_dmidecode.txt`

Serial numbers, BIOS version and the DIMM in each slot.

8. `ip a > unit123_ip.txt` (and `nvidia-smi -q > unit123_gpu.txt` on a GPU server)

Network and GPU state.

Then copy the files off the server before the power cycle, for example `scp unit123_* tech@10.1.1.50:/evidence/`, and note the time and what was seen on the console.

REMEMBER IT

Save before you reboot: Kernel, Journal, BMC, Hardware. Name files with the serial, copy them off.

► EXPLANATION (understand it)

A power cycle wipes the dmesg buffer, the live sensor readings and the current state of every device. If you do not save them, the engineer gets a server that "works fine now" and no way to find the cause.

Ten minutes is plenty: each command takes seconds. Copying the files off the server matters because if the disk is part of the problem, files saved only on it may be lost too.

SEE IT: TOPOLOGY / DIAGRAM



Name every file with the unit serial. Copy the files OFF the server before the power cycle.

Ten minutes is enough to save the evidence that would otherwise be lost: kernel messages, the journal, BMC events and sensors, the device list and the serials.

> TRY IT: EXAMPLE COMMAND OUTPUT (assumed values, to show what each command prints)

Saving everything (seconds each)

```
$ dmesg -T > unit123_dmesg.txt
$ journalctl -b > unit123_journal.txt
$ sudo ipmitool sel elist > unit123_sel.txt
$ sudo ipmitool sensor > unit123_sensors.txt
$ sudo lspci -vv > unit123_lspci.txt
$ sudo dmidecode > unit123_dmi decode.txt
$ ls -lh unit123_*
-rw-r--r--. 1 tech tech 88K Oct 2 09:21 unit123_dmesg.txt
-rw-r--r--. 1 tech tech 4.1M Oct 2 09:21 unit123_journal.txt
-rw-r--r--. 1 tech tech 12K Oct 2 09:21 unit123_sel.txt
-rw-r--r--. 1 tech tech 9.6K Oct 2 09:21 unit123_sensors.txt
```

→ Each file is a snapshot of one kind of evidence.

Copy it off the server

```
$ scp unit123_* tech@10.1.1.50:/evidence/J7K2Q41/
unit123_dmesg.txt          100% 88KB 31.2MB/s 00:00
unit123_journal.txt        100% 4196KB 101.4MB/s 00:00
...
```

→ Now the evidence survives even if the server never comes back.

Memory hooks: one-page review

1	What servers do	Servers = 24/7, nobody watching. So they get R-E-H-R: Redundancy, ECC, Hot-swap, Remote management.
2	What PCIe is for	PCIe = the CPU's highways to the cards. Lanes = traffic lanes: more lanes, more data at once.
3	Retimers versus redrivers	Redriver = megaphone (louder, noise too). Retimer = translator: listens, then says it again clearly.
4	DHCP messages and settings	DORA: Discover, Offer, Request, Acknowledge. Settings: "I Must Get DNS Later" = IP, Mask, Gateway, DNS, Lease.
5	ESD risks	ESD does 3 things: Kill, Weaken, Glitch. Prevent with Strap, Mat, Bag, Edges.
6	POST	POST = the server's morning check-up: CPU, Memory, PCIe, ROMs, Boot. Memory training is the long part.
7	SSH versus Serial Over LAN	SSH talks to the OS. SOL talks to the console through the BMC. OS broken = SSH gone, SOL still works.
8	BIOS and UEFI settings	BIOS settings: Boot, PCIe, Memory, CPU/Power (+ Console). "Big PCs Make Commotion".
9	ECC memory	ECC: 1 bit = fix it, 2 bits = flag it (stop). A rising corrected count = replace soon.
10	Server will not power on	Follow the power: Outlet, Cord, PSU, Standby/BMC, Button, Board. The BMC tells you if standby power arrives.
11	Isolation techniques	Four ways to catch the bad part: Swap, Strip, Split, Compare.
12	Slow NVMe drive	Half speed? Check Link, Path, Heat, Firmware, Format, Test, Health. Then swap bays.
13	Failure follows the part	Follows the part = bad part. But it does not prove Server A is OK.
14	ip a shows 169.254	169.254 = "I asked DHCP and nobody answered." The link is fine; the path to DHCP is not.
15	Memory: DIMM, board, CPU, or configuration	Map it first: which slots, which channels, which DIMMs are new. Then: Install, DIMM, Slot, CPU.
16	Replacement did not fix it	New part, same failure, same slot = not the part. Stop swapping; start measuring.
17	Filtering large logs	-b this boot, -p err errors, --since/--until a time window, sort uniq -c sort -rn top repeats.
18	Network card checks by command	Four steps up: Found (lspci), Driver (lspci -k), Link (ethtool), Address (ip a).
19	First commands on an unknown server	Who am I on? What hardware? What is wrong now? What did the BMC see? How is it connected? Look before you touch.
20	Slow server: processes and load	Compare load average to cores. Then look at %us, %wa, available, swap. Busy CPU, short memory, or waiting disk?
21	What Linux is and why servers use it	Linux = kernel; distro = kernel + tools. Reasons: S-L-R-T ("Servers Love Running Tux").
22	Reading ip a output	Read left to right: Name, Flags, MTU, MAC, IP/mask, brd, dynamic. Not shown: Gateway, DNS, Speed.
23	How a name becomes an IP address	hosts first, then resolv.conf, then the DNS server, then cache. A record = name to IPv4.
24	PCIe bandwidth and the math	GT/s x lanes / 8 = GB/s each way. Gen 4 x8 = 16 x 8 / 8 = 16 GB/s.
25	Topology tree versus diagram	Diagram = expected. Tree (lspci -tv) = actual. The shape of what is missing points to the part.
26	What the BMC is and does	BMC = own CPU, own network port, standby power. Does: SEL, Sensors, Inventory, KVM, SOL, Power, NMI, Boot/ISO.

27	LnkCap versus LnkSta	Cap = Can. Sta = Status (what it got). Sta lower than Cap = downtrained: works, but slower.
28	The four layers of a working device	P-B-D-F: Physical, Bus, Driver, Function ("Please Buy Donuts Friday"). Find the first layer that fails.
29	Symptom versus root cause	Symptom = what you see. Root cause = why. Ask "why?" until you reach a process.
30	Server lost network after a move	First: is it alive? Ask the BMC. Then bottom-up: Link, NIC, IP, Gateway, Name, Service.
31	New card missing from lspci	Walk the whole path: Search, BIOS/BMC, Seat, Settings, Tree, Logs, Swap. Only then call it dead.
32	Intermittent failure	Log every run, find the pattern, then cause it on purpose. Loop + watch + one change at a time.
33	Powers on but no OS	Bootting is a chain: Power, POST, BIOS, GRUB, Kernel, Startup. Watch the console and find where it stops.
34	NVMe drops out under load	A reboot hides it, it does not fix it. Evidence: kernel log, SEL, smart-log, LnkSta, firmware. Then swap bays.
35	GPU missing, empty port in lspci -tv	Empty port = the GPU end (GPU, power cable, seating, slot, retimer). Missing branch = upstream.
36	Assembly cable error on three units	V-D-N-C-P + CAPA: Verify, Document, Notify, Contain, Prevent. Fix the process, not the person.
37	grep across a folder	grep -rin = Recursive, Ignore case, line Numbers. Count: -c or wc -l.
38	dmesg versus journalctl -b -1	dmesg = whiteboard (wiped every reboot). journalctl -b -1 = notebook (last boot is still there).
39	IP works but name fails, and the reverse	IP works, name fails = DNS. Name works, IP fails = network or target.
40	Saving evidence before a reboot	Save before you reboot: Kernel, Journal, BMC, Hardware. Name files with the serial, copy them off.